MadStree: formula handbook

2026-10-22

Abstract

This handbook documents the formula system of `MadStree`. The package handles arbitrary mixed massive/massless de Sitter tree graphs. Rather than generating a general system of IBP equations, it assembles the master integrals, the iterative reduction and the block-triangular dlog-form differential equation directly from the topology; the public interface is listed in Section 13. The purely massive formulas of [1] admit a uniform description as Kronecker sums of one-dimensional theta-free exponential systems and two-dimensional Hankel systems. Each vertex carries a single exponential parameter $k_{0,v}$, whose generator $\mathrm{i}p_{0,v}$ is the tensor product of that one-dimensional factor with the identity on all remaining factors. A massless (anti-)time-ordered propagator with theta is not a single theta-free exponential factor: before reduction it carries one two-dimensional Hankel state at each endpoint, four states in total. The massless 00/11 and 10/01 relations compress these four states by a constant projection into one two-dimensional state shared by the whole propagator. This factor acts at both endpoints in the time-IBP, so a vertex-by-vertex tensor factorization fails, although the tensor product and Kronecker sum over the full graph remain valid. Single-vertex IBPs, iteration matrices and dlog diagonal blocks then follow directly from the topology; theta derivatives only add fixed off-diagonal blocks that connect sectors with a pinched propagator, without a prior large IBP system. Quantities quoted verbatim from [1] carry the label unreduced: they are rewritten in the $+\nu$, positive-prefactor convention of this handbook but do not include the massless reduction above; see Section 2. Time-only sectors are identified by fixed-length strings in the order of the root propagators.

Contents

1 Topology input convention **1**
1.1 Loading and citation 2

2 Scope and conventions **2**

3 Local function systems: one-dimensional exponentials and two-dimensional h **4**
3.1 One-dimensional systems of theta-free exponentials 4
3.2 The two-dimensional h system 4
3.3 Positive/negative prefactor conventions and the massless exponential kernel 5

4 Kronecker-sum structure of purely massive vertices **5**

5 From vertex spaces to the full-graph space **7**

6 Massless (anti-)time-ordered propagators: from four states to a shared two-state **7**
6.1 It is not a one-dimensional exponential factor 7
6.2 Two vertices act on the same two-dimensional factor 8
6.3 The contact selector is also obtained directly from the projection 9

7 Full-graph single-vertex IBP with massless (anti-)time-ordered propagators 9
7.1 Positive/negative vertices, SK signs and Wick coordinates 11

8 Direct iterative reduction 12
8.1 The [1] binary order and the domain of the recurrences 12
8.2 Two-directional recurrences including the contact remaining term 13

9 The full-graph tensor formula for the dlog DE 17
9.1 Bit deletion, insertion and the massless quotient in Eq. (3.68) of [1] 19

10 The dimensionless h equation, physical quadratic-momentum terms and realification 20

11 Interface with the package conventions 21
11.1 The massless quotient basis of the package 21
11.2 The unique quotient and its four-state derivation space 23
11.3 $(-\tau)^A$ and normalized-J 23
11.4 Single-vertex families, local inverses and the complete reduction interface 24
11.5 The small-time end of the integration path and the safe time-IBP shift 26

12 Pure-exponential-energy infinity boundary and blow-up for arbitrary time-only graphs 26
12.1 Damping variables and auxiliary one-dimensional exponentials 26
12.2 Strict total time order, nested blow-up and theta fixing 27
12.3 Choosing the preferred rank from a user-preferred ordinary point 27
12.4 Contact subsectors inherit the same root rank 28
12.5 Controlled transport of an auxiliary external-leg energy to zero 30
12.6 The [2] infinity boundary and FlintNDE transport 30
12.7 Symbolic Laurent-support certificate and regulator error control 37

13 Public interface reference 39

14 Usage examples 41
14.1 Single-vertex family: definition, reduction, and unified point input 41
14.2 Two-vertex massless (anti-)time-ordered propagator: multiple sectors and points . 42
14.3 Massive three-vertex tree: zero middle external-leg energy is not automatically singular 42
14.4 Three-vertex regulator: automatic Laurent support and incremental fitting 42
14.5 No physical external leg: private auxiliary energy to zero 43

15 Minimal algorithm for direct formula generation and its acceptance tests 43

16 Conclusions 44

1 Topology input convention

Every vertex sets its Schwinger–Keldysh contour branch through `vertexType -> "+"|"-"`. The propagator fields `skType`, `sigma`, and the endpoint signs are not entered separately; public `type` accepts only `"massive"` or `"massless"`. A three-vertex chain reads

```
<|
 "vertices" -> {
   <|"id"->1,"externalLegEnergy"->k1,"timePower"->a1,"vertexType"->"+"|>,
   <|"id"->2,"externalLegEnergy"->k2,"timePower"->a2,"vertexType"->"+"|>,
```

```
  <|"id"->3,"externalLegEnergy"->k3,"timePower"->a3,"vertexType"->"-"|>
 },
 "lines" -> {
  <|"type"->"massless","endpoints"->{1,2},"momentum"->q12|>,
  <|"type"->"massive","endpoints"->{2,3},"momentum"->q23,"nu"->nu23|>
 }
|>
```

The ordered types of the two vertices determine the T/W/External class. T denotes the time-ordered (Feynman) component G_{++} and the anti-time-ordered (anti-Feynman) component G_{--}, collectively the (anti-)time-ordered propagators, and W denotes the Wightman components G_{+-} and G_{-+}. These classes also determine the `++/--/+-/-+` label, the T sign, the signs at the two vertices, and default Hankel branches. Names such as `masslessT` and σ_e in formulas (the $G_{\pm\pm}$ label of a propagator, defined in Sec. 6) are derived internal physical data, not duplicated user fields. Vertex IDs may be integers. The public propagator interface neither defines nor reads an ID and are always numbered internally by their exact `lines` input positions; only an explicit `thetaBundles` declaration uses these position numbers. The fields above serve two initialization entries: `MSInitTree[spec]` accepts a tree topology, and `MSInitTimeGraph[spec]` accepts a time-only graph with cycles (no loop momentum or ISP); both return the same kind of context and share every downstream interface.

The field `externalLegEnergy` is the theta-free external-leg exponent parameter attached to vertex v; it does not mean that the vertex itself owns an energy. `vertexType=+` gives $\exp(-\mathrm{i}k_v\tau_v)$ and `vertexType=-` gives $\exp(+\mathrm{i}k_v\tau_v)$. The SK contour sign and this exponent sign are stored separately internally, but both are derived uniquely from the same `vertexType`. For a vertex without a physical external leg, `externalLegEnergy` may be omitted or set explicitly to zero. Both forms normalize to zero in the topology; the boundary producer uses a private auxiliary damping energy to construct the infinity boundary and transports that auxiliary coordinate back to the user-specified zero.

1.1 Loading and citation

The standard Wolfram entry point prints one citation notice per kernel only after every module has loaded successfully. The arXiv labels are clickable in a notebook, while a headless kernel keeps the complete URLs. The Python file protocol does not mix a text notice into JSON stdout; it stores the same MadStree citation list in response metadata, while direct adapter CLI use prints the complete text. Research using MadStree should cite:

1. Jiaqi Chen and Bo Feng, "Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations", arXiv:2401.00129.

2. Jiaqi Chen, Bo Feng and Yi-Xiao Tao, "Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations", arXiv:2411.03088.

3. Jiaqi Chen, Yi-Xiao Tao, Xinyi Wang and Dong Zhang, "MadStree: automated evaluation of tree-level de Sitter correlators", arXiv:2610.28526.

2 Scope and conventions

The formulas are taken from Eqs. (3.1)–(3.5), (3.37), (3.47), (3.50), (3.54)–(3.55) and (3.64)–(3.68) of [1]. That reference uses the vertex time power τ^{ν_0} (the subscript 0 marks the time-power layer and ν_0 is that time power) and

$$h(\nu,0;z)=z^{-\nu}H_\nu(z),\qquad h(\nu,1;z)=\partial_z h(\nu,0;z),\qquad z=-k\tau, \tag{1}$$

Here ν is the Hankel order of the propagator, $a \in \{0,1\}$ is the state bit of the mode function (0 is h itself and 1 is its first z derivative), k is the momentum modulus of the propagator, τ is the time of the vertex to which it is attached, and $z = -k\tau$. That reference also uses the natural binary basis. MadStree uses the input parameter ν as-is as the Hankel order: ν may be real, purely imaginary $\mathrm{i}\mu$ (principal series/heavy fields), or complex, and no layer of the formulas and no part of the program ever takes an absolute value or a real part of ν. The prefactor convention is carried by a single sign s_ν, whose subscript ν marks that it adjusts the Hankel-order power of z; the package uses only one function name h:

$$\boxed{h(\nu,0;z) = z^{s_\nu \nu} H_\nu(z)}, \qquad s_\nu \in \{+1,-1\}. \tag{2}$$

The default `NuConvention -> "Positive"` corresponds to $s_\nu = +1$; `"Negative"` corresponds to $s_\nu = -1$, which is the convention adopted by the printed formulas of refs. [1] and [2]. In both conventions the Hankel order is always the input ν itself, and the switch happens only in the powers of z. The two definitions do not differ merely by a constant; they differ by $z^{2\nu}$. Since the Hankel equation contains only ν^2, the two sets of dimensionless equations of motion read respectively

$$h'' + \frac{1-2\nu}{z} h' + h = 0 \quad (s_\nu = +1), \qquad h'' + \frac{1+2\nu}{z} h' + h = 0 \quad (s_\nu = -1). \tag{3}$$

Thus the two sets of algebraic formulas differ strictly only by $s_\nu \nu \to -s_\nu \nu$ (written below in the customary shorthand as $\nu \to -\nu$). This notation is only the algebraic consequence of flipping the sign of s_ν; it does not replace the Hankel order by a conjugate order: what the integrand stores is always H_ν. The positive-prefactor basis may also be compared with the same function taken at a negative parameter in the convention of [1], $h(-\nu,0;z) = z^\nu H_{-\nu}(z)$. Within one Hankel branch, $H^{(\mathrm{b})}_{-\nu}$ and $H^{(\mathrm{b})}_{\nu}$ differ only by a kinematics-independent branch phase: $H^{(\mathrm{b})}_{-\nu} = e^{\mathrm{i}\sigma_\mathrm{b}\pi\nu} H^{(\mathrm{b})}_{\nu}$, where $\mathrm{b} = 1,2$ take $\sigma_\mathrm{b} = +1,-1$, respectively. This constant absorbs the order difference entirely into the normalization. For a purely imaginary order $\nu = \mathrm{i}\mu$ the second branch gives exactly $e^{-\pi\mu} H^{(2)}_{-\mathrm{i}\mu} = H^{(2)}_{\mathrm{i}\mu}$ of Eq. (167) in Section 11. Let ν_ref denote the parameter of the same name in the printed formulas of that paper; the subscript ref marks that printed convention. It differs from the ν used in this handbook and in the default positive-prefactor implementation of MadStree only by

$$\boxed{\nu_\mathrm{ref} = -\nu}. \tag{4}$$

Converting to the printed formulas of that paper through Eq. (4) requires more than renaming the Hankel subscript: every ν in the equations of motion, the Wronskian, M_1 (the time-power-layer matrix defined in Eqs. (17)–(18)), the contact powers, the recurrences and the dlog is replaced simultaneously by ν_ref. The replacement only rewrites the same integrand in the printed convention and leaves the stored Hankel order unchanged. Equation (4) is the only prefactor conversion in this handbook; every formula below is already written in it and therefore uses the $+\nu$, positive-prefactor convention exclusively. The base formula stored in the program is this $s_\nu = +1$, $+\nu$ form: every order slot in a formula is read as `formulaNu` $:= s_\nu\, \nu$. With the default positive prefactor `formulaNu`$= +\nu$, and with `"Negative"` `formulaNu`$= -\nu = \nu_\mathrm{ref}$, matching the printed formulas of that paper verbatim. In the package context `hankelOrder` always stores the input ν itself, while `formulaNu` and `hPrefactorPower` equal $s_\nu \nu$. Section 11 gives the adaptation to the package's $(-\tau)^A$ (with A the explicit time power at each vertex), normalized-J ($J_s = \mathcal{N}_s \mathrm{I}_s$, with $\mathcal{N}_s$ the sector normalization and its subscript s marking the sector) and sector normalization.

The main difference from [1] is the massless bulk propagator. The general iterative reduction Eqs. (3.37), (3.47), (3.54)–(3.55) and the subsector Eqs. (3.64)–(3.68) of that paper are derived for a massive pinched propagator, which carries an independent two-dimensional h state at each endpoint. That paper notes in Eqs. (3.13)–(3.14) that the massless $G_{\pm\pm}$ also obeys vanishing and pinched relations, and gives in Section 3.6 the 0-fold vertex attached only to massless propagators, with $2^0 = 1$ master integral; it does not, however, use these relations to compress the four states

at the two endpoints of a massless (anti-)time-ordered propagator into one two-dimensional state shared by the whole propagator. Accordingly, every quantity quoted verbatim from that paper carries the label unreduced, meaning "the formula of [1], already written in the $+\nu$ convention of this handbook via Eq. (4), but not yet specialized by the massless reduction relations". The 00/11 and 10/01 relations of Section 6 and the $4 \to 2$ projection of Eq. (30) are precisely these extra relations; inserting them into the unreduced quantities gives the final formulas of this handbook.

Two classes of objects are distinguished:

- a pure exponential whose direction has already been chosen in an external leg or in G_{+-}/G_{-+} carries no theta and is only a one-dimensional function system;
- the massless (anti-)time-ordered propagator in G_{++}/G_{--} contains two terms and a theta. Before reduction, like the massive (anti-)time-ordered propagator, it has one two-dimensional h state at each of its two vertices, four states in total.

The core result is that the massless extra relations compress the four states of the second class into one shared two-dimensional factor, rather than treating it as two independent one-dimensional exponentials.

The conclusions fall into two tiers. The single-vertex matrices of a fixed sector, the four-to-two-state projection and the common diagonalization are direct algebraic results. The injection of contact terms into lower sectors can likewise be written directly, but the constant-residue dlog and flatness under arbitrary mixed/common-theta normalization still require independent verification. The derivation above does not replace those checks.

3 Local function systems: one-dimensional exponentials and two-dimensional h

3.1 One-dimensional systems of theta-free exponentials

Each vertex v carries exactly one theta-free exponential parameter, namely the input field `externalLegEnergy`, written $k_{0,v}$ in this handbook (the subscript 0 marks the one-dimensional exponential layer of the vertex and v is the vertex label); its exponential factor is

$$E_v(\tau_v) = \exp(\mathrm{i}\varsigma_v k_{0,v}\tau_v)\,, \qquad \varsigma_v = \pm 1, \tag{5}$$

where ς_v is the vertex exponent sign, with the subscript v marking its vertex, and is derived uniquely from `vertexType`; see Section 1 and Eq. (53). The single user symbol $k_{0,v}$ takes as its value the sum of the momentum moduli of all theta-free external legs attached to v; Eq. (3.1) of [1] likewise writes only one k_0 per vertex, which is the same summed quantity. From here on this handbook always uses the summed $k_{0,v}$, and introduces neither per-leg nor per-block exponential parameters nor any summation over them. Viewing E_v as a vector in the one-dimensional space $\mathcal{E}_v \simeq \mathbb{C}$, its time generator and signed energy are

$$Q_v = [\mathrm{i}\varsigma_v k_{0,v}], \qquad \mathbb{I}_{\mathcal{E}_v} = [1], \qquad p_{0,v} = \varsigma_v k_{0,v}, \qquad \mathrm{Emb}_v(Q_v) = \mathrm{i}p_{0,v}\mathbb{I}. \tag{6}$$

Here Q_v is the one-dimensional time generator at vertex v, $\mathbb{I}_{\mathcal{E}_v}$ is the identity on that one-dimensional factor, $p_{0,v} = \varsigma_v k_{0,v}$ is the signed energy (with the same meaning of $0, v$), and Emb_v embeds the local factor into the full graph space, as defined in Eq. (16). A one-dimensional exponential therefore does not increase the number of master integrals and does not create a slot: a vertex corresponds to a single one-dimensional factor $\mathcal{E}_v$, M_1 does not contain it, and M_0 contains only the scalar $\mathrm{i}p_{0,v}\mathbb{I}$. A massless Wightman/external branch also has no theta; it does not create a separate one-dimensional factor either, and the momentum moduli of its vertices enter the same pure-exponential energy of the component according to Eq. (58). Every Hankel derivative term is also multiplied by the identity matrix of the one-dimensional factor, so all single-vertex time derivatives remain Kronecker-sum terms of the same kind.

3.2 The two-dimensional h system

All formulas below use the $+\nu$, positive-prefactor convention. Let

$$\boldsymbol{h}(\nu;z)=\begin{pmatrix}h(\nu,0;z)\\ h(\nu,1;z)\end{pmatrix},\qquad P_1=\begin{pmatrix}0&0\\0&1\end{pmatrix},\qquad \sigma_2=\begin{pmatrix}0&-\mathrm{i}\\ \mathrm{i}&0\end{pmatrix}. \tag{7}$$

Here $\boldsymbol{h}$ collects the two state bits into a column vector, P_1 projects onto the state bit $a=1$, and σ_2 is the second Pauli matrix. The Hankel branch superscript is suppressed: $h(\nu,a;z)$ denotes either branch, $h^{(1)}(\nu,a;z)$ or $h^{(2)}(\nu,a;z)$, in the notation $h^{(\alpha)}(\nu,a;z)$ of the paper, where $\alpha\in\{1,2\}$ labels the Hankel branch; the superscript is written only where the two branches are distinguished. Eqs. (3.2)–(3.4) of [1] are equivalent, in the convention of this handbook, to

$$\partial_\tau\boldsymbol{h}=\left[-\mathrm{i}k\sigma_2-\frac{1-2\nu}{\tau}P_1\right]\boldsymbol{h}. \tag{8}$$

Here $-\mathrm{i}k\sigma_2$ is a time-independent 2×2 matrix, and $-(1-2\nu)P_1/\tau$ merges with the lowering of the time power into M_1.

The same system in the dimensionless variable $z=-k\tau$ satisfies

$$\partial_z^2h(\nu,0;z)+\frac{1-2\nu}{z}\partial_zh(\nu,0;z)+h(\nu,0;z)=0. \tag{9}$$

The constant term here is 1, not k^2. The k is recovered only through

$$\partial_\tau=-k\partial_z \tag{10}$$

This layering allows the matrices of [1] to be reused below.

3.3 Positive/negative prefactor conventions and the massless exponential kernel

The Hankel equation depends only on the square of the order, but here the Hankel order is always the input ν as-is, and the two conventions only switch the powers on $z^{\pm\nu}$; Eq. (4) is the only conversion between the two prefactors and is not repeated below. In the default positive-prefactor convention of MadStree ($s_\nu=+1$, `formulaNu`$=+\nu$), when $\nu=1/2$,

$$h(1/2,0;z)=z^{1/2}H^{(1,2)}_{1/2}(z)\propto e^{\pm\mathrm{i}z},\qquad (1-2\nu)|_{\nu=1/2}=0. \tag{11}$$

Consequently

$$\partial_z^2h(1/2,0;z)+h(1/2,0;z)=0, \tag{12}$$

and the first derivative state is the branch eigenstate of the zeroth-order state,

$$h^{(1)}(1/2,1)=+\mathrm{i}h^{(1)}(1/2,0),\qquad h^{(2)}(1/2,1)=-\mathrm{i}h^{(2)}(1/2,0). \tag{13}$$

The negative-prefactor option gives $z^{-1/2}H^{(1,2)}_{1/2}\propto z^{-1}e^{\pm\mathrm{i}z}$, which is no longer a pure exponential; its formulas are obtained from the $+\nu$ base formulas by switching the order slot `formulaNu` from $+\nu$ to $-\nu$. This is only the algebraic consequence of flipping the sign of s_ν: the integrand still stores H_ν; within one branch $H^{(\mathrm{b})}_{-\nu}$ differs from $H^{(\mathrm{b})}_{\nu}$ only by the branch constant $e^{\mathrm{i}\sigma_\mathrm{b}\pi\nu}$ ($\mathrm{b}=1,2$ with $\sigma_\mathrm{b}=+1,-1$), with no interchange of branches. For a purely imaginary order $\nu=\mathrm{i}\mu$ one especially cannot say that the order has been replaced by a conjugate order: within one branch the two differ only by a constant, $H^{(1)}_{-\mathrm{i}\mu}=e^{-\pi\mu}H^{(1)}_{\mathrm{i}\mu}$ and $H^{(2)}_{-\mathrm{i}\mu}=e^{+\pi\mu}H^{(2)}_{\mathrm{i}\mu}$; the second is exactly $e^{-\pi\mu}H^{(2)}_{-\mathrm{i}\mu}=H^{(2)}_{\mathrm{i}\mu}$ of Eq. (167). For a complex or purely imaginary order, a logarithm branch must be chosen first for $z^{\pm\nu}$; `Abs[nu]` or taking the real part of ν is never called anywhere.

4 Kronecker-sum structure of purely massive vertices

Consider the p-fold vertex integral of [1]

$$\mathrm{I}_v(\nu_{0,v},\boldsymbol{a}) = \int \mathrm{d}\tau_v\, \tau_v^{\nu_{0,v}} e^{\mathrm{i}p_{0,v}\tau_v} \prod_{i=1}^{p} h(\nu_i,a_i;-k_i\tau_v), \qquad a_i\in\{0,1\}. \tag{14}$$

Here I_v is the integral of vertex v, the subscript v is the vertex label, p is the number of massive legs at that vertex, $\nu_{0,v}$ is the time power of the vertex, $\boldsymbol{a}=(a_1,\dots,a_p)$ is the multi-index of leg state bits, ν_i, a_i and k_i are the Hankel order, state bit and momentum modulus of leg i, τ_v is the vertex time, and $p_{0,v}$ is the signed energy of Eq. (6). Its local space is

$$\mathcal{V}_v = \mathcal{E}_v \otimes \bigotimes_{i=1}^{p} \mathcal{H}_i, \qquad \dim\mathcal{E}_v = 1, \quad \dim\mathcal{H}_i = 2. \tag{15}$$

Here $\mathcal{V}_v$ is the local space of vertex v, $\mathcal{E}_v$ is its one-dimensional exponential factor, $\mathcal{H}_i$ is the two-dimensional Hankel space of leg i, and $\mathbb{I}_j$ is the identity on factor j.

For any local factor $j=1,\dots,N$ (with N the total number of factors), define the embedding

$$\mathrm{Emb}_j(A) = \mathbb{I}_1\otimes\cdots\otimes A_j\otimes\cdots\otimes\mathbb{I}_N. \tag{16}$$

Then a single time IBP of the integrand is performed: ∂_{τ_v} acts only on the factors belonging to vertex v—the one-dimensional exponential E_v of that vertex and the p h blocks of the massive propagators attached to it—while the factors of the other vertices do not move and carry only the identity matrix; Eq. (3.37) of [1] is therefore written directly in the convention of this handbook as

$$\mathrm{M}_{1,v}(\nu_{0,v}) = \nu_{0,v}\mathbb{I}_v - \sum_{i=1}^{p}(1-2\nu_i)\,\mathrm{Emb}_i(P_1), \tag{17}$$

$$\mathrm{M}_{0,v} = \mathrm{i}p_{0,v}\mathbb{I}_v + \sum_{i=1}^{p}\mathrm{Emb}_i(-\mathrm{i}k_i\sigma_2). \tag{18}$$

$\mathrm{M}_{1,v}$ and $\mathrm{M}_{0,v}$ are the time-power-layer and momentum-layer matrices of vertex v: the subscript 1 marks the layer multiplying the lowered vector, the subscript 0 marks the layer multiplying the vector at the same power, and v marks the vertex. $\mathbb{I}_v$ is the identity on the local space of that vertex. Using $P_1=(\mathbb{I}-\sigma_3)/2$, the first equation becomes the Λ_3 form of that paper, written in the convention of this handbook,

$$\mathrm{M}_{1,v} = \sum_i\left(\frac{1}{2}-\nu_i\right)\Lambda_3^{(i)} + \left(\nu_{0,v}-\frac{p}{2}+\sum_i\nu_i\right)\mathbb{I}_v. \tag{19}$$

Here $\Lambda_3=\sigma_3$ and $\Lambda_2=\sigma_2$ are the Pauli-matrix notation of that paper, and the superscript (i) indicates that the matrix acts only on the two-dimensional factor of leg i. Thus $p_0\mathbb{I}$ and $k_i\Lambda_2^{(i)}$ are not essentially different: the former is the generator of the one-dimensional exponential factor of that vertex, the latter comes from the two-dimensional h factors; both are local generators multiplied by all the spectator identities.

Let $\boldsymbol{f}_v(a)$ denote the 2^p-dimensional integral vector obtained after shifting $\nu_{0,v}$ by the integer a. Ignoring the theta remaining term,

$$\mathrm{M}_{1,v}(\nu_{0,v}+a)\boldsymbol{f}_v(a-1) + \mathrm{M}_{0,v}\boldsymbol{f}_v(a) = 0. \tag{20}$$

One lowering step is directly

$$A_{-,v}(a) = -\mathrm{M}_{1,v}(\nu_{0,v}+a)^{-1}\mathrm{M}_{0,v}. \tag{21}$$

Here $A_{\pm,v}(a)$ is the one-step raising/lowering matrix of vertex v at shift a: the subscript + means raising, − means lowering, v marks the vertex, and a is the integer shift of $\nu_{0,v}$.

The constant matrix of [1]

$$\mathrm{T} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -\mathrm{i} \\ -\mathrm{i} & 1 \end{pmatrix}, \qquad \mathrm{T}\sigma_2\mathrm{T}^{-1} = \sigma_3, \tag{22}$$

simultaneously diagonalizes all two-dimensional local generators in M_0; $\mathrm{T}_v = \mathrm{T}^{\otimes p}$ is its tensor product over the p two-dimensional factors of vertex v. Then

$$\widetilde{\mathrm{M}}_{0,v} = \mathrm{T}_v \mathrm{M}_{0,v} \mathrm{T}_v^{-1} \tag{23}$$

is diagonal, where the tilde marks a quantity after the change to the T basis, and one raising step is

$$A_{+,v}(a) = -\mathrm{T}_v^{-1} \widetilde{\mathrm{M}}_{0,v}^{-1} \mathrm{T}_v \mathrm{M}_{1,v}(\nu_{0,v} + a + 1). \tag{24}$$

So the purely massive formulas never need a general large-matrix inversion from the outset: M_1 is diagonal in the original basis, and $\widetilde{\mathrm{M}}_0$ is diagonal in the T basis.

5 From vertex spaces to the full-graph space

Let V_s be the set of components of a fixed sector s; its members are called the vertices of the sector: a component merged by contacts is one vertex, integrated over a single time, and in the top sector V_s is the vertex set of the graph. Before the massless extra relations are used, the h state at each vertex of a propagator is an independent two-dimensional factor, and each vertex carries one one-dimensional exponential factor. The single-vertex full-graph space can therefore be written as

$$\mathcal{V}_s^{\text{unreduced}} = \bigotimes_{v \in V_s} \mathcal{E}_v \otimes \bigotimes_{\text{all active two-state factors } i} \mathcal{H}_i. \tag{25}$$

A one-dimensional factor does not create a slot and contributes only the scalar $p_{0,v}$. The $\mathrm{M}_{0,v}$ and $\mathrm{M}_{1,v}$ of every vertex are embedded into this common space by Eq. (16). In the time IBP with respect to τ_v, ∂_{τ_v} acts only on the building blocks belonging to that vertex: the one-dimensional exponential factor of the vertex, the h blocks of the massive propagators attached to it, and, for theta-free Wightman propagators, the pure exponential branch in which it participates as a vertex of the propagator; the factors of the other vertices carry only the identity matrix. For a purely massive graph, the single-vertex operator of vertex v is therefore supported on its own local factors, so the vertex families of different vertices factorize independently. The theta derivatives of G_{++}/G_{--} do not belong to the single-vertex operator; they are added separately as lower-sector remaining terms.

This full-graph formulation is more fundamental than writing the formula once for each vertex. The latter is valid only because, in the purely massive case, the single-vertex operators of different vertices are supported on mutually disjoint two-dimensional factors.

6 Massless (anti-)time-ordered propagators: from four states to a shared two-state

6.1 It is not a one-dimensional exponential factor

Consider a massless (anti-)time-ordered propagator $e = (u_e, v_e)$ containing theta, where e is the propagator label and u_e, v_e are its first and second endpoints. Before the extra relations are used, it is exactly like a massive propagator: the vertex u_e has two states $a \in \{0, 1\}$, the vertex v_e has two states $b \in \{0, 1\}$, so the local space of the propagator is

$$\mathcal{H}_{e,u} \otimes \mathcal{H}_{e,v} \simeq \mathbb{C}^4, \tag{26}$$

where $\mathcal{H}_{e,u}$ and $\mathcal{H}_{e,v}$ are the two-dimensional h spaces at the two endpoints. In the natural order of ref. [1] take

$$\mathrm{I}_e^{(4)} = (\mathrm{I}_{00}, \mathrm{I}_{01}, \mathrm{I}_{10}, \mathrm{I}_{11})^T, \tag{27}$$

where the superscript (4) marks the four-state vector and the two subscripts are the state bits at u_e and v_e. A massless (anti-)time-ordered propagator has two theta chambers, and the two Hankel branches occupy them in opposite orders. Name the two undifferentiated chambers A_e and B_e (the subscript e marks the propagator): for G_{++}, A_e carries $h^{(1)}$ at u_e and $h^{(2)}$ at v_e, B_e carries the reversed pair, and G_{--} interchanges the two names. Then $\mathrm{I}_{00} = A_e + B_e$, and by Eq. (13) the τ_{u_e} derivative multiplies A_e by $+\mathrm{i}$ and B_e by $-\mathrm{i}$, while the τ_{v_e} derivative does the reverse. In the dimensionless h-state convention of ref. [1] this gives

$$\mathrm{I}_{01} = -\mathrm{I}_{10}, \qquad \mathrm{I}_{11} = \mathrm{I}_{00}. \tag{28}$$

The two chambers are different integrals with complementary time orderings, so no constant relates I_{10} to I_{00}: at exchange-symmetric kinematics $A_e = B_e$, where I_{10} vanishes and I_{00} does not. The reduction is therefore $4 \to 2$, not $4 \to 1$, and the shared factor below really is two-dimensional. No k^2 is missing here: each I_{ab} uses $h_1 = -k^{-1}\partial_\tau h_0$, not the physical derivative at a vertex from which k has not been removed. It is also not the algebraic quotient of the package, which factors out the phase of the derivative at each vertex; the 11 relation of the latter carries a minus sign. The physical relation is restored in Sec. 10, and the package quotient is given in Sec. 11.

Choose the shared two-state

$$\boldsymbol{g}_e = (\mathrm{I}_{00}, \mathrm{I}_{10})^T \in \mathcal{M}_e \simeq \mathbb{C}^2. \tag{29}$$

The constant embedding and projection between the four states and the two states are

$$\boxed{\mathrm{I}_e^{(4)} = S_e \boldsymbol{g}_e, \qquad S_e = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P_e = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad P_e S_e = \mathbb{I}_2.} \tag{30}$$

Here $\boldsymbol{g}_e = (\mathrm{I}_{00}, \mathrm{I}_{10})^T$ is the shared two-state vector of the propagator and $\mathcal{M}_e$ is its space; S_e embeds the two states into the four, P_e projects the four back to two, and the subscript e marks the propagator. Any original four-dimensional operator O_e that preserves the relation subspace can be reduced directly to

$$O_e^{\mathrm{red}} = P_e O_e S_e, \tag{31}$$

where the superscript red marks the reduced operator.

6.2 Two vertices act on the same two-dimensional factor

Before reduction, the constant h generator of the vertex u_e is $-\mathrm{i}q_e\sigma_2 \otimes \mathbb{I}_2$, and that of the vertex v_e is $-\mathrm{i}q_e\mathbb{I}_2 \otimes \sigma_2$, where q_e is the momentum modulus of propagator e. Applying Eq. (30) directly gives

$$P_e(\sigma_2 \otimes \mathbb{I}_2)S_e = \sigma_2, \tag{32}$$

$$P_e(\mathbb{I}_2 \otimes \sigma_2)S_e = -\sigma_2. \tag{33}$$

Therefore, once the four states are reduced to two, the two vertices of the propagator no longer each have an independent tensor factor; instead they act with $+\sigma_2$ and $-\sigma_2$ respectively on the same $\mathcal{M}_e$. Within the single-vertex structure, this is precisely the only reason for the failure of vertex-by-vertex factorization.

Importantly, the full-graph tensor-product structure does not fail. What fails is the decomposition

$$\mathcal{V}_s \stackrel{?}{=} \bigotimes_{v\in V_s} \mathcal{V}_v, \tag{34}$$

because the shared $\mathcal{M}_e$ cannot belong independently to both u_e and v_e; but all vertex operators are still Kronecker sums on the same full-graph space.

6.3 The contact selector is also obtained directly from the projection

Separating the overall scalar from the four-state remaining-term pattern of Eq. (3.65) of [1], the pattern is

$$\boldsymbol{r}_e^{(4)} = (0, -1, +1, 0)^T. \tag{35}$$

It satisfies exactly

$$\boldsymbol{r}_e^{(4)} = S_e|1\rangle_e, \qquad |1\rangle_e = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \tag{36}$$

So contact does not need to be re-solved: in the shared two-state it is simply the fixed column vector $|1\rangle_e$. The overall scalar $w_e^{(v)}$ depends on the two-state basis, its full notation being $w_{e,\mathrm{basis}}^{(v)}$: the subscript e marks the propagator, the second subscript marks the two-state basis (the h instance here), and the superscript marks the endpoint vertex. Let $\sigma_e = +1, -1$ denote G_{++}, G_{--} respectively for propagator e; in the h-state basis of ref. [1] used in this section, the contact weights of the two vertices u_e and v_e, taken in that order, are

$$w_{e,h}^{(u_e)} = +2\mathrm{i}\sigma_e, \qquad w_{e,h}^{(v_e)} = -2\mathrm{i}\sigma_e. \tag{37}$$

The constant basis change of Sec. 11 converts these exactly into the package's $-2, +2$. One only needs, in addition, to multiply by the common-theta odd-subset coefficient and the sector-normalization ratio; a massless propagator carries no extra Hankel Wronskian prefactor.

7 Full-graph single-vertex IBP with massless (anti-)time-ordered propagators

Fix a contact-reachable sector s and let V_s be its set of components, whose members are the vertices of the sector (defined in Section 5). Write

- $H(s)$ for all still-active h slots of massive propagators;
- $E_m(s)$ for all unpinched massless (anti-)time-ordered propagators.

The argument s marks the sector and m marks massless; $H_v(s)$ is the subset of $H(s)$ attached to vertex v. The reduced full-graph space is

$$\boxed{\mathcal{V}_s = \bigotimes_{v\in V_s} \mathcal{E}_v \otimes \bigotimes_{i\in H(s)} \mathcal{H}_i \otimes \bigotimes_{e\in E_m(s)} \mathcal{M}_e.} \tag{38}$$

Since $\dim \mathcal{E}_v = 1$, its dimension is

$$\dim \mathcal{V}_s = 2^{|H(s)|+|E_m(s)|}. \tag{39}$$

This gives a slot registry that can be initialized directly. Each slot records a stable number, its type, the root propagator or root vertex it belongs to, the user momentum name, and the basis and SK/Hankel metadata; each root vertex v additionally stores its incidence list

$$\mathcal{B}(v) = \{i \in H : v(i) = v\} \cup \{(e, \epsilon_{ve}) : e \in E_m,\ v \in e\}. \tag{40}$$

$\mathcal{B}(v)$ is the incidence list of vertex v; H and E_m abbreviate $H(s)$ and $E_m(s)$, and ϵ_{ve} is the incidence of propagator e at vertex v, with subscript v marking the vertex and e the propagator. One-dimensional exponentials do not enter this registry: they occur in M_0 only as the scalars $p_{0,v}$, whereas the user still supplies numerical points and differentiation variables in terms of their own $k_{0,v}$ and the propagator moduli q_e; the two coordinate sets are related by the pullback of Eq. (57). The same label e of a massless (anti-)time-ordered propagator appears in the lists of both vertices; this is exactly the data expression of "shared, not copied."

For each propagator, whose two vertices are ordered as $e = (u_e, v_e)$, define the incidence

$$\epsilon_{ve} = \begin{cases} +1, & v = u_e, \\ -1, & v = v_e, \\ 0, & v \notin e. \end{cases} \tag{41}$$

In the h-state basis of ref. [1], the choice of Hankel branch changes the particular solution but not the matrix $-\mathrm{i}q_e\sigma_2$ of the first-order system. Therefore the branch sign of G_{++}/G_{--} is not written into the generator here; that sign appears explicitly when changing to the package quotient basis.

The contribution of each kind of atom to a vertex v can be written uniformly as follows. The superscript marks the atom type: E_v is the one-dimensional exponential of the vertex, h_i the h block of massive leg i, and m_e the massless shared block of propagator e; the subscript v or u_e, v_e marks the vertex on which the atom acts.

$$\Delta\mathrm{M}^{(E_v)}_{1;v} = 0, \qquad \Delta\mathrm{M}^{(E_v)}_{0;v} = \mathrm{i}p_{0,v}\mathbb{I}, \qquad v \in V_s, \tag{42}$$

$$\Delta\mathrm{M}^{(h_i)}_{1;v} = -(1-2\nu_i)P_1, \qquad \Delta\mathrm{M}^{(h_i)}_{0;v} = -\mathrm{i}k_i\sigma_2, \qquad i \in H_v, \tag{43}$$

$$\Delta\mathrm{M}^{(m_e)}_{1;u_e} = \Delta\mathrm{M}^{(m_e)}_{1;v_e} = 0, \qquad \Delta\mathrm{M}^{(m_e)}_{0;u_e} = -\mathrm{i}q_e\sigma_2, \qquad \Delta\mathrm{M}^{(m_e)}_{0;v_e} = +\mathrm{i}q_e\sigma_2. \tag{44}$$

The ν_i in Eq. (43) are already the parameters of the $+\nu$ convention of this handbook; the base formula stored inside the program is exactly this $s_\nu = +1$, $+\nu$ form, where the order slot reads `formulaNu` $= s_\nu\nu_i$, i.e. $+\nu_i$ under the default positive prefactor, so the $(1 - 2\,$`formulaNu`$)$ of the code agrees directly with the $(1-2\nu_i)$ of this handbook. When comparing verbatim with the printed formulas of that paper, replace the order by $\nu_{\mathrm{ref}} = -\nu_i$ through Eq. (4); the $(1-2\nu_i)$ and $(2\nu_i - 1)$ of this handbook then reproduce the printed $(2\nu_{\mathrm{ref}} + 1)$ and $(-2\nu_{\mathrm{ref}} - 1)$ of that paper. The global M_1 also receives the component time power $\nu_{0,v}\mathbb{I}$. The two vertices u_e and v_e of Eq. (44) act on the same two-dimensional slot and cannot each multiply out a new slot.

On the full-graph space (38), the single-vertex matrices of each vertex are directly

$$\boxed{\mathrm{M}_{1;v,s}(\nu_{0,v}) = \nu_{0,v}\mathbb{I}_s - \sum_{i\in H_v(s)} (1-2\nu_i)\,\mathrm{Emb}_i(P_1),} \tag{45}$$

$$\boxed{\mathrm{M}_{0;v,s} = \mathrm{i}p_{0,v}\mathbb{I}_s + \sum_{i\in H_v(s)} \mathrm{Emb}_i(-\mathrm{i}k_i\sigma_2) + \sum_{e\in E_m(s)} \mathrm{Emb}_e(-\mathrm{i}\epsilon_{ve}q_e\sigma_2).} \tag{46}$$

A massless propagator does not enter M_1: the default $\nu = 1/2$ makes the $1-2\nu$ of Eq. (11) vanish identically. The same $\mathrm{Emb}_e(\sigma_2)$ appears in both $\mathrm{M}_{0;u_e,s}$ and $\mathrm{M}_{0;v_e,s}$ at the two vertices of the propagator, with opposite incidence signs.

All $\mathrm{M}_{0;v,s}$ are made of σ_2 on different slots, or of the same σ_2 on the same shared slot, so they commute with one another and can be simultaneously diagonalized by a single global constant transformation

$$U_s = \bigotimes_{i\in H(s)} \mathrm{T}_i \otimes \bigotimes_{e\in E_m(s)} \mathrm{T}_e \tag{47}$$

simultaneously; the subscript s marks the sector, i a massive slot, and e a massless shared slot; the one-dimensional exponential factors only contribute [1] and are not written explicitly.

This step does not mean that arbitrary Pauli combinations can be handled this way. For any two-dimensional slot α, write the local part of each vertex v on that slot as

$$Q_{v\alpha} = c_{0,v\alpha}\mathbb{I} + \boldsymbol{c}_{v\alpha}\cdot\boldsymbol{\sigma}, \tag{48}$$

where the subscript v marks the vertex and α the two-dimensional factor; $c_{0,v\alpha}$ is the scalar part (subscript 0 marks the zeroth order of the Pauli expansion) and $\boldsymbol{c}_{v\alpha}$ is the vector of the three Pauli coefficients. The condition for a common diagonalizer to exist slot by slot is

$$[Q_{v\alpha}, Q_{w\alpha}] = 0 \quad\Longleftrightarrow\quad \boldsymbol{c}_{v\alpha}\times\boldsymbol{c}_{w\alpha} = 0 \qquad (\text{all } v, w). \tag{49}$$

That is, all Pauli directions on the same slot must be collinear. In the h-state of ref. [1] at present, only σ_2 appears on every massive/shared slot; in the package quotient, massive slots carry only σ_2 and each massless shared slot carries only σ_1, so the global diagonalizer is changed to

$$U_s^J = \bigotimes_{i\in H(s)} \mathrm{T}_i \otimes \bigotimes_{e\in E_m(s)} H_e, \qquad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{50}$$

Here the superscript J marks the normalized-J basis and H_e is the Hadamard matrix acting on the shared slot e. If, in the future, a shared slot simultaneously receives non-collinear σ_1 and σ_2, a single 2×2 matrix might still diagonalize it separately, but different vertices would no longer share a common U_s; in that case one must not continue replacing the large-matrix inversion with a uniform letter-by-letter reciprocal.

Let the massive bits be $a_i \in \{0,1\}$ and the bits of the shared massless-propagator slots be $r_e \in \{0,1\}$. Then

$$U_s \mathrm{M}_{0;v,s} U_s^{-1} = \mathrm{diag}(\mathrm{i}\kappa_{v,s;\boldsymbol{a},\boldsymbol{r}})\,, \tag{51}$$

where

$$\boxed{\kappa_{v,s;\boldsymbol{a},\boldsymbol{r}} = p_{0,v} + \sum_{i\in H_v(s)} (2a_i - 1)k_i + \sum_{e\in E_m(s)} \epsilon_{ve}(2r_e - 1)q_e.} \tag{52}$$

Here $\kappa_{v,s;\boldsymbol{a},\boldsymbol{r}}$ is the diagonal letter of vertex v in sector s, with massive bits $\boldsymbol{a}$ and shared bits $\boldsymbol{r}$: the subscript v marks the vertex, s the sector, and the two multi-indices after the semicolon collect the massive and massless-shared state bits. The same r_e enters the letters of both u_e and v_e with opposite signs. This is the complete single-vertex content of the correlation carried by the shared slot.

The massless quotient therefore does not prevent the use of Eqs. (45)–(46) for a single vertex: given v, one only needs to add the terms block by block from $\mathcal{B}(v)$. The meaning of this vertex-by-vertex assembly is stated here explicitly: in the time IBP with respect to τ_v, ∂_{τ_v} acts only on the building blocks belonging to that vertex—the one-dimensional exponential factor of the vertex, the h blocks of the massive propagators attached to it, and the massless shared blocks in which it participates as a vertex of the propagator (with incidence $\epsilon_{ve} = \pm 1$); the factors of the remaining vertices carry only the identity matrix, so Eqs. (45) and (46) are exactly the Kronecker sum of that vertex's local atoms. What changes is not this vertex-by-vertex assembly rule but the support relation between the matrices of different vertices: $\mathrm{M}_{0;u_e,s}$ and $\mathrm{M}_{0;v_e,s}$ contain opposite coefficients on the same slot e, rather than acting on two unrelated slots. The two vertex-by-vertex recurrences share a state bit, yet each is still a closed-form matrix formula; there is no need to assemble a large system of IBP equations first.

7.1 Positive/negative vertices, SK signs and Wick coordinates

The theta-free exponential of Eq. (5) is unique per vertex and reads

$$\exp(\mathrm{i}\varsigma_v k_{0,v}\tau_v), \qquad \varsigma_v = +1 \text{ or } -1, \tag{53}$$

where ς_v is derived uniquely from `vertexType`: `"+"` gives $\varsigma_v = -1$ and `"-"` gives $\varsigma_v = +1$. ς_v is the vertex phase sign; it is neither the G_{++}/G_{--} label σ_e of a massless propagator nor the incidence ϵ_{ve} fixed by the order of the two vertices of the propagator.

The only effect of positive and negative vertices on the single-vertex formulas is

$$p_{0,v} = \varsigma_v k_{0,v}, \qquad \mathrm{M}_{0;v,s} \supset \mathrm{i} p_{0,v} \mathbb{I}_s. \tag{54}$$

M_1 does not contain the one-dimensional exponential; the P_1, σ_2 atoms of the massive h are likewise unchanged by the Hankel branch. The difference between G_{++} and G_{--} is instead stored in the theta branch, the Wronskian/contact weights, and in D_{σ_e} of the package quotient; it cannot be replaced by ς_v.

Now set $\tau_v = -t_v < 0$, where t_v is the damping time of vertex v. Along the damping rays of the two kinds of vertices define uniformly

$$K_v = \mathrm{i}\varsigma_v k_{0,v} > 0, \qquad k_{0,v} = -\mathrm{i}\varsigma_v K_v. \tag{55}$$

Then

$$\exp(\mathrm{i}\varsigma_v k_{0,v} \tau_v) = \exp(-K_v t_v). \tag{56}$$

Therefore, for $\varsigma_v = +1$ one may write $k_{0,v} \to -\mathrm{i}\, pik_{0,v}$ in the reference notation, while for $\varsigma_v = -1$ one writes $k_{0,v} \to +\mathrm{i}\, mik_{0,v}$; in both cases $pik_{0,v}$ and $mik_{0,v}$ are positive. The main text uses these names only for communication with the literature; internally, the program should generate the stable K_v coordinates at initialization.

The user still supplies points with their own $k_{0,v}$ and all propagator moduli q_e from initialization. When the connection is output in user coordinates, only the invertible pullback

$$\mathrm{d}p_{0,v} = \varsigma_v \mathrm{d}k_{0,v}, \qquad \mathrm{d}K_v = \mathrm{i}\varsigma_v \mathrm{d}k_{0,v}; \tag{57}$$

is performed; the user's q_e are not silently renamed `pik` or `mik`. When contact merges a set of root vertices into a component C, the pure-exponential energy and the damping variable of that component are

$$p_{0,C} = \sum_{u \in C} \varsigma_u k_{0,u} + \sum_{\substack{e \text{ massless Wightman/external} \\ v \in e \cap C}} (-\varsigma_v)\, q_e, \qquad K_C = \sum_{u \in C} K_u. \tag{58}$$

Here the subscript C marks the component formed by contact. The second term comes from the theta-free massless Wightman/external branches: such a branch creates no slot, and only adds the momentum moduli of those of its vertices that lie in C, each with the SK contour sign $-\varsigma_v$ of that vertex, into the same scalar; the sum runs over active propagators only. The first term alone is $p_{0,u}$, so for a single-vertex component, Eq. (58) reduces to the $p_{0,v}$ of Eq. (54) exactly when that vertex carries no theta-free propagator branch.

8 Direct iterative reduction

8.1 The [1] binary order and the domain of the recurrences

Fix a sector s, let n_s^{slot} be the number of two-state factors in the sector and c_s the number of time components after contact. Following the binary representation of [1], write a state as

$$\boldsymbol{a} = (a_1, \ldots, a_{n_s^{\mathrm{slot}}}), \qquad a_i \in \{0, 1\}. \tag{59}$$

The state order is fixed to

$$\boldsymbol{a}(m) = \mathrm{IntegerDigits}(m, 2, n_s^{\mathrm{slot}}), \qquad 0 \le m < 2^{n_s^{\mathrm{slot}}}. \tag{60}$$

The first bit is the most significant; in a Kronecker product the left slots vary slowly and the right slots vary quickly. For any bit string, $\mathrm{Del}_i\,\boldsymbol{a}$ deletes the i-th bit and $\mathrm{Ins}_i(\widehat{\boldsymbol{c}}, x)$ inserts $x \in \{0,1\}$ at position i. These conventions are exactly in the same order as `IntegerDigits[...,2,n]`, `Delete` and `Insert` of the reference code.

A native integral of MadStree can be written as

$$J_s(\boldsymbol{n};\boldsymbol{a}) \equiv \texttt{MSIntegral[}s,\boldsymbol{n},\boldsymbol{a}\texttt{]}, \qquad \boldsymbol{n} \in \mathbb{Z}^{c_s}, \quad \boldsymbol{a} \in \{0,1\}^{n_s^{\mathrm{slot}}}. \tag{61}$$

Here the subscript s marks the sector, $\boldsymbol{n}$ is the c_s-component integer time-shift vector (its v-th entry shifts the local time power $\nu_{0,v}$ of vertex v), $\boldsymbol{a}$ is the n_s^{slot}-component state-bit vector. The sector label s is not recovered from the local $\boldsymbol{n}, \boldsymbol{a}$. Let the root propagators be, in the initialization order, $E = (e_1, \ldots, e_{N_E})$, where N_E is the number of root propagators, and let the pinched canonical set be L_s^{ctr}. Define the fixed-length string

$$\mathrm{key}(s) = q_1 q_2 \cdots q_{N_E}, \qquad q_j = \begin{cases} 0, & e_j \in L_s^{\mathrm{ctr}}, \\ 1, & e_j \notin L_s^{\mathrm{ctr}}. \end{cases} \tag{62}$$

Propagators that cannot be pinched always have $q_j = 1$, and the top sector is the all-1 string. The key is a string, not a binary integer; leading zeros cannot be dropped. For example, if $E = (e_1, e_2, e_3, e_4)$ and $L_s^{\mathrm{ctr}} = \{e_1, e_2, e_4\}$, the key is `"0010"`. The full integral head always puts this key in the first argument. Hence two subsectors remain different `MSIntegral` expressions even if $c_s, n_s^{\mathrm{slot}}, \boldsymbol{n}, \boldsymbol{a}$ are all identical. If the same pinch set is reached through different contact paths, it is merged into the same sector. Before returning the context, `MSInitTree` generates a `sectorIdentityCertificate` that checks in one pass the canonical pinch sets, the sector keys, the full master expressions, the consecutive global indices, and the SHA-256 digest of the complete ordered master table; any collision returns `SectorIdentityCollision`, so that no ambiguity is left to the recurrence or DE consumers.

Thus the input domain of `MSReduce` is not "arbitrary dS integrals" but the space spanned, in a fixed initialization context, by the following legal objects

$$\boxed{\mathrm{span}\Big\{ J_s(\boldsymbol{n};\boldsymbol{a}) \,\Big|\, s \in \mathrm{ContactDAG},\ \boldsymbol{n} \in \mathbb{Z}^{c_s},\ \boldsymbol{a} \in \{0,1\}^{n_s^{\mathrm{slot}}} \Big\}.} \tag{63}$$

That is, one may choose arbitrary integer time shifts and all binary states in each contact-reachable sector, and take finite linear combinations of such legal objects; this set of recurrences cannot introduce new propagator powers, new discrete indices, sectors outside the context, or another function family.

8.2 Two-directional recurrences including the contact remaining term

Let $\mathrm{I}_s(\boldsymbol{n})$ denote the ordered master vector of sector s, where n_v shifts the local time power $\nu_{0,v}$ of vertex v itself—the v-th slot of `aShifts` in the native `MSIntegral`. The theta remaining term is written as a linear combination of lower-sector vectors

$$\mathcal{R}_{v,s}(\boldsymbol{n}) = \sum_{t \prec s} C_{st}^{(v)}(\boldsymbol{n}) \mathrm{I}_t(\boldsymbol{n}_t). \tag{64}$$

Here $\mathcal{R}_{v,s}(\boldsymbol{n})$ is the remaining term of vertex v in sector s: the subscript v marks the differentiated vertex and s the sector; $C_{st}^{(v)}$ is its coefficient into the lower sector t, with the superscript v marking the source vertex. More concretely, for an ordinary single-propagator contact it can be written as

$$\mathcal{R}_{v,s}(\boldsymbol{n}) = \sum_{\substack{e \in E_{\mathrm{T}}(s) \\ v \in e}} C_{s,s/e}^{(v,e)} \mathrm{I}_{s/e}(\chi_e(\boldsymbol{n}))\,. \tag{65}$$

Here $E_{\mathrm{T}}(s)$ is the set of all (anti-)time-ordered propagators in sector s, s/e denotes the subsector obtained by pinching propagator e, and the sub/superscripts of $C^{(v,e)}_{s,s/e}$ mark the source sector/subsector and the vertex–propagator pair participating in that contact. χ_e (with the subscript e marking the propagator) merges the time shifts belonging to the two vertices of the propagator and includes the fixed shift by which the delta contact makes the subsector base power one less. This supersector-to-subsector structure is identical for massive and massless propagators; only the local matrices $C^{(v,e)}$ differ. A massive propagator uses the complementary state bits and signs at its two vertices, given by Eq. (3.65) of [1]; a massless propagator first takes the $4 \to 2$ quotient and then uses the shared-bit selector of Eq. (78) together with the $-2, +2$ of Eq. (119). Hence massless pinch does not need a second set of recurrence formulas.

The package fixes the subsector base power deliberately according to the delta contact: $\mathcal{R}(\mathbf{0})$ lands on subsector shift -1, while the $\mathcal{R}(\boldsymbol{e}_v)$ needed by the DE lands exactly on subsector shift 0, where $\boldsymbol{e}_v$ is the unit time-shift vector with a 1 in the v-th entry. Below, the remaining term used by the raising step is denoted $R^{(1)} := \mathcal{R}_{v,s}(\boldsymbol{e}_v)$ and the one used by the lowering step $\mathcal{R}_{v,s}(\mathbf{0})$ is denoted $R^{(0)}$; they are the two remainders of the same one-step relation evaluated at different indices, their targets differ by one subsector shift, and they are not interchangeable. Hence $R^{(1)}$ of an ordinary single-propagator massive/massless contact already hits the subsector master directly; there is no problem of "a dlog master being chosen but the iterative reduction failing to reach it."

The complete one-step IBP at vertex v is

$$\mathrm{M}_{1;v,s}(\nu_{0,v} + n_v)\mathrm{I}_s(\boldsymbol{n} - \boldsymbol{e}_v) + \mathrm{M}_{0;v,s}\mathrm{I}_s(\boldsymbol{n}) + \mathcal{R}_{v,s}(\boldsymbol{n}) = 0. \tag{66}$$

Eq. (66) is written vertex by vertex: each vertex v has its own one-step relation, and that relation is exactly what the time IBP ∂_{τ_v} of that vertex produces. ∂_{τ_v} acts only on the building blocks belonging to vertex v—the one-dimensional exponential factor of the vertex, the h blocks of the massive propagators attached to it, and the massless shared blocks in which it participates as a vertex of the propagator—while the factors of the other vertices carry only the identity matrix. Therefore $\nu_{0,v}$, $\mathrm{M}_{1;v,s}$, $\mathrm{M}_{0;v,s}$, $\mathcal{R}_{v,s}$ and the later $A_{\pm,v,s}$ all read only the local atoms of vertex v, and the shift directions $\boldsymbol{e}_v$ do not mix between different vertices: the iterative reduction of the whole family proceeds one vertex at a time, and `MSReduce` likewise applies the relation of each component separately.

One lowering step is therefore directly

$$\mathrm{I}_s(\boldsymbol{n} - \boldsymbol{e}_v) = A_{-,v,s}(\boldsymbol{n})\mathrm{I}_s(\boldsymbol{n}) - \mathrm{M}^{-1}_{1;v,s}\mathcal{R}_{v,s}(\boldsymbol{n}), \tag{67}$$

$$A_{-,v,s}(\boldsymbol{n}) = -\mathrm{M}_{1;v,s}(\nu_{0,v} + n_v)^{-1}\mathrm{M}_{0;v,s}. \tag{68}$$

One raising step uses the diagonal matrix

$$\widetilde{\mathrm{M}}_{0;v,s} = U_s \mathrm{M}_{0;v,s} U_s^{-1} \tag{69}$$

and gives

$$\mathrm{I}_s(\boldsymbol{n} + \boldsymbol{e}_v) = A_{+,v,s}(\boldsymbol{n})\mathrm{I}_s(\boldsymbol{n}) - U_s^{-1}\widetilde{\mathrm{M}}^{-1}_{0;v,s}U_s\mathcal{R}_{v,s}(\boldsymbol{n} + \boldsymbol{e}_v), \tag{70}$$

$$A_{+,v,s}(\boldsymbol{n}) = -U_s^{-1}\widetilde{\mathrm{M}}^{-1}_{0;v,s}U_s\mathrm{M}_{1;v,s}(\nu_{0,v} + n_v + 1). \tag{71}$$

These are the direct generalizations of Eqs. (3.37), (3.47), (3.50) and (3.66) of ref. [1] to the space with shared massless-propagator slots. What still needs to be inverted is only the diagonal M_1 in the original basis and the diagonal $\widetilde{\mathrm{M}}_0$ in the U_s basis. The complete singular layers at general indices can now be obtained directly.

Before explaining the singular layers, one must distinguish the h block in the formulas from the complete physical mode. If d is the spatial dimension and contributions such as the curvature coupling are absorbed into m_{eff}, the scalar index satisfies

$$\nu^2 = \frac{d^2}{4} - \frac{m^2_{\mathrm{eff}}}{H^2}. \tag{72}$$

Four-dimensional de Sitter has $d = 3$, and the complete mode contains

$$u_k(\tau) \propto (-\tau)^{3/2} H_\nu^{(1,2)}(-k\tau), \tag{73}$$

rather than the Hankel function alone. For general d the two late-time branches are

$$u_k(\tau) \sim c_-(-\tau)^{d/2-\nu} + c_+(-\tau)^{d/2+\nu}. \tag{74}$$

For a minimally coupled massless scalar, $\nu = d/2$, and the leading branch is exactly the zeroth power while the other branch is the d-th power. For a light field with $0 < m_{\text{eff}}^2 < d^2H^2/4$, one has $0 < \nu < d/2$ and both branches have positive real part; for a heavy field $\nu = \mathrm{i}\mu$, both real parts equal $d/2 > 0$. Hence the mass term indeed enters Eq. (72) with a minus sign, and the natural late-time powers of a generic massive mode do not by themselves produce boundary divergences.

$h = z^{\pm\nu} H_\nu$ only redistributes the $\pm\nu$ power of Eq. (74) between the explicit $(-\tau)^A$ and the interior of the block; the same convention must be fixed after initialization, but the complete physical power is unchanged. If the master integrals keep the natural $(-\tau)^{d/2}$ prefactor of each propagator, both massive branches decay at $\tau \to 0$. Although the massless leading branch is a constant, a total-derivative primitive used by the IBP still tends to zero as long as it carries one more positive explicit $(-\tau)$ power; what is genuinely dangerous is the total primitive power dropping to zero or below. This corresponds exactly to the layers below where the M_1 eigenvalue vanishes or is crossed; one must not add extra boundary terms merely because "the massless mode is itself a constant," nor continue using the boundary-discarding recurrence on the zero layer.

For a state $(\boldsymbol{a}, \boldsymbol{r})$, the M_1 eigenvalue is

$$\lambda_{1;v,\boldsymbol{a}}(n_v) = \nu_{0,v} + n_v - \sum_{i \in H_v} (1 - 2\nu_i) a_i, \tag{75}$$

which is independent of the massless shared bits $\boldsymbol{r}$ and of all kinematics. When lowering $n_v \to n_v - 1$, only

$$\lambda_{1;v,\boldsymbol{a}}(n_v) = 0 \tag{76}$$

produces an M_1^{-1} pole. A general mixed/Hankel family still needs to be judged against the concrete small-τ asymptotics; but a purely massless vertex has no terms from massive propagators, and setting $\alpha_v = \nu_{0,v} + n_v$ gives $\mathrm{M}_{1;v} = \alpha_v \mathbb{I}$, while the corresponding integral at the small-time end indeed contains $\int_0^\infty \mathrm{d}t_v\, t_v^{\alpha_v - 1} e^{-\kappa t_v} = \Gamma(\alpha_v)\kappa^{-\alpha_v}$. Hence when the whole M_1 vanishes at $\alpha_v = 0$, the unregularized Gamma integral itself diverges and the boundary-discarding IBP cannot be used directly. One should first take $\alpha_v \to \alpha_v + \delta$ as analytic regularization; in the suitably regularized generic-δ system M_1 is invertible, and after the recurrences are completed one expands and takes $\delta \to 0$, so no physical failure of the recurrences appears. For massive states, if one instead uses the natural time powers without the absorbed $H \to h$ prefactor, a group of m equal orders with r of them having $a_i = 1$ contributes $r + (m - 2r)\nu$; generically, different orders do not cancel systematically, and the ν of the group cancels only when m is even and $r = m/2$. For such M_1 zeros one likewise first moves them away with an analytic regulator in the time power and then runs the recurrences at generic regulator.

In the opposite direction, raising $n_v \to n_v + 1$, the denominator is

$$\lambda_{0;v,\boldsymbol{a},\boldsymbol{r}} = \mathrm{i}\kappa_{v,s;\boldsymbol{a},\boldsymbol{r}}, \qquad \kappa_{v,s;\boldsymbol{a},\boldsymbol{r}} = 0. \tag{77}$$

Here $\lambda_{0;v,\boldsymbol{a},\boldsymbol{r}}$ is the diagonal eigenvalue of $\mathrm{M}_{0;v,s}$: the subscript 0 marks the momentum layer, v the vertex, and the entries after the semicolon are the state bits. This is the kinematics energy-letter singular surface. Here k_i and q_e are momentum moduli; ordinary vector momentum conservation does not force κ to vanish. When the relevant momenta degenerate to collinear and split into two opposite groups L, R along the axis, momentum conservation becomes $\sum_{\ell \in L} |\boldsymbol{k}_\ell| - \sum_{\ell \in R} |\boldsymbol{k}_\ell| = 0$, which is exactly $\kappa = 0$ for the corresponding choice of state signs. In the raising formula, $\mathrm{M}_1 = 0$ appears only in the numerator and produces no pole; in the lowering formula, $\mathrm{M}_0 = 0$ likewise

appears only in the numerator. If both vanish, the row cannot be crossed in either direction and the formulas leave only a lower-sector constraint; one then needs another relation or a different basis, but one still cannot declare that a new master necessarily appears merely from the rank loss. The program should return the direction, shift, state bits, the zero eigenvalue and the original undivided IBP row, without hiding these layers behind a fixed maximum number of steps.

The 0-fold special case is the clearest: $\mathrm{M}_1 = \nu_0 + n$ and $\mathrm{M}_0 = \mathrm{i}p_0$, where ν_0 is the time power of that vertex, n its integer time shift and p_0 its signed energy. Lowering fails at $\nu_0 + n = 0$ and raising fails at $p_0 = 0$; these are precisely the two directions to which the two denominators of Eq. (3.59) of [1] belong, not one "unified singular layer."

For a single massless propagator e pinched to $t = s/e$, Eq. (36) directly gives

$$\boxed{C^{(v,e)}_{s,s/e} = w^{(v)}_e \,\mathrm{PermuteToGlobalOrder}\left(|1\rangle_e \otimes \mathbb{I}_{\mathrm{spectators}}\right).} \tag{78}$$

If a bundle of (anti-)time-ordered propagators $\mathcal{B}$ shares the same theta argument, they are not a product of independent deltas. Sharing means that the two vertices of each of the $|\mathcal{B}| \geq 2$ (anti-)time-ordered propagators lie in the same two current components, so their theta arguments are all the same $\Delta = \tau_u - \tau_v$; this is exactly the criterion by which the program groups the eligible (anti-)time-ordered propagators into bundles. Write

$$\prod_{e\in\mathcal{B}} G_e = \theta(\Delta)\prod_{e\in\mathcal{B}} A_e + \theta(-\Delta)\prod_{e\in\mathcal{B}} B_e, \qquad \mathsf{J}_e = \frac{A_e + B_e}{2}, \quad D_e = A_e - B_e. \tag{79}$$

Eq. (79) only multiplies out the thetas, and no parity enters at this step: within one bundle $\theta(\Delta)$ is the same function, and since $\theta(\Delta)^2 = \theta(\Delta)$ and $\theta(\Delta)\theta(-\Delta) = 0$, every mixed term vanishes, so the product of $|\mathcal{B}| = n$ propagators leaves only two terms instead of 2^n. The 2^n terms arise only for propagators with pairwise different theta arguments, and those propagators belong to different bundles; the contact events are performed bundle by bundle.

Parity comes from the next step, differentiating the shared theta. The whole bundle has only one θ, so a single differentiation gives only one $\delta(\Delta)$,

$$\partial_\Delta\left[\theta(\Delta)\prod_e A_e + \theta(-\Delta)\prod_e B_e\right] = \delta(\Delta)\left(\prod_e A_e - \prod_e B_e\right), \tag{80}$$

that is, the coefficient of δ is exactly the antisymmetric difference under $A \leftrightarrow B$, in which all the contact information is concentrated; the two vertices are also forced to coincide only once by this single δ, and the components merge only once, independently of how many propagators are selected.

The third step telescopes the antisymmetric difference and then changes basis. The telescoping identity is

$$\begin{aligned}\prod_{e=1}^{n} A_e - \prod_{e=1}^{n} B_e &= \sum_{k=1}^{n}\left(A_1\cdots A_k B_{k+1}\cdots B_n - A_1\cdots A_{k-1}B_k\cdots B_n\right)\\ &= \sum_{k=1}^{n} D_k \prod_{e<k} A_e \prod_{e>k} B_e,\end{aligned} \tag{81}$$

and each term contains exactly one D_k. Substituting $A_e = \mathsf{J}_e + D_e/2$ and $B_e = \mathsf{J}_e - D_e/2$ for the spectators and collecting by the number of D factors gives

$$\begin{aligned}\prod_e A_e - \prod_e B_e &= \sum_{S\subseteq\mathcal{B}}\left(1-(-1)^{|S|}\right)2^{-|S|}\prod_{e\in S} D_e \prod_{e\notin S}\mathsf{J}_e\\ &= \sum_{\substack{\varnothing\neq S\subseteq\mathcal{B}\\ |S|\ \text{odd}}} 2^{1-|S|}\prod_{e\in S} D_e\prod_{e\notin S}\mathsf{J}_e.\end{aligned} \tag{82}$$

The parity can also be read off from symmetry alone: the left-hand side of Eq. (81) changes sign under $A \leftrightarrow B$, while D_e changes sign and J_e does not, so only terms containing an odd number of D factors can survive; the coefficient $1-1$ at $|S|=0$ removes the empty set automatically, and the same holds for every even $|S|$. In the coefficient $2^{1-|S|}$, the factor $2^{-|S|}$ comes from the $D_e/2$ of each selected propagator and the extra factor 2 from $1-(-1)^{|S|}$ on odd subsets.

The program uses the collected form (82) rather than the term-by-term telescoping (81): the k-th telescoping term corresponds to "pinching only the k-th propagator," but after the spectators are expanded the same S receives contributions from several k, so enumerating the telescoping terms would count the same sector transitions repeatedly. After the collection each event appears once and its coefficient is a pure number; in the implementation this means taking the nonempty odd subsets of each bundle, recording the coefficient as $2^{1-|S|}$, and deduplicating by source sector, target sector and the selected propagator ids.

Hence an odd subset S is a simultaneous contact event: the selected propagators are pinched at once, their two vertices merge only once, and the contact selector is the tensor product of the local selectors of the selected propagators times $2^{1-|S|}$. If the two vertices of an unselected massless (anti-)time-ordered propagator become coincident after merging, its odd shared state is zero; an unselected massive (anti-)time-ordered propagator uses the equal-time canonical quotient $10 \sim 01$. The program first builds the matrices in the raw binary space and then projects with $P_s \mathrm{M}^{\mathrm{raw}} S_s$ onto the true sector masters, so the coincident quotient does not require a general large-matrix inversion.

For an event-reachable component C, the subsector base power of time can be written directly from the final partition as

$$\nu_{0,C} = \sum_{u \in C} \nu_{0,u} + \sum_{e \in E_s(C)} \varrho_e + |C| - 1, \tag{83}$$

where ϱ_e is the raw contact power of the pinched propagator. The last term $|C|-1$ is the number of vertex merges that actually occur; a common-theta event of k propagators contributes once, not the erroneous k times. Eq. (83) depends only on the final component and the pinched set, so different legal contact paths give the same base power. Every event strictly reduces the number of components, so the sector BFS terminates automatically, and a repeated contact of an active propagator whose two vertices already lie in one component is excluded.

9 The full-graph tensor formula for the dlog DE

Define the diagonal matrix-valued logarithm of each vertex

$$\widetilde{\Omega}_{0;v,s} = \mathrm{diag}_{\boldsymbol{a},\boldsymbol{r}} \left[-\mathrm{i} \log \kappa_{v,s;\boldsymbol{a},\boldsymbol{r}} \right]. \tag{84}$$

Here v marks the vertex, s the sector and 0 the EOM zero layer; the tilde marks a quantity already in the U_s-diagonal basis, whose diagonal entries are labelled by the massive bits $\boldsymbol{a}$ and the shared bits $\boldsymbol{r}$. The explicit momentum terms of [1] come only from the slots of massive propagators, which are still two-dimensional h systems, and the subscript ex marks the explicit momentum term while s marks the sector:

$$(\Omega_{\mathrm{ex},s})_{\boldsymbol{b}\boldsymbol{a}} = \delta_{\boldsymbol{b}\boldsymbol{a}} \left[-\sum_{i \in H(s)} a_i (1-2\nu_i) \log k_i \right]. \tag{85}$$

A shared slot of a massless propagator does not contribute this term, because the default positive prefactor ($s_\nu = +1$, `formulaNu`$= \nu = 1/2$) makes $1-2\nu = 0$; under the negative prefactor the $\nu = 1/2$ massless propagator is no longer a pure exponential, which was already stated separately after Eq. (11).

If a massless propagator is instead assigned the next degenerate half-integer order $\nu = 3/2$, the result still lies inside the same family; no new slot, atom, or master is required. From the explicit spherical Hankel functions

$$z^{3/2}H^{(1)}_{3/2}(z) = (1-\mathrm{i}z)\,z^{1/2}H^{(1)}_{1/2}(z), \qquad z^{3/2}H^{(2)}_{3/2}(z) = (1+\mathrm{i}z)\,z^{1/2}H^{(2)}_{1/2}(z), \tag{86}$$

the $\nu = 3/2$ block exceeds the pure-exponential $\nu = 1/2$ block of Eq. (11) only by a first-degree polynomial in z. After $z = -k\tau$ that polynomial becomes a first-degree polynomial in τ: the constant term reproduces the original $\nu = 1/2$ block; the τ term merely raises the explicit $(-\tau)^A$ power by one, which the package records in the `aShifts` of the corresponding vertex without introducing a new function type; the accompanying momentum constant $(\pm\mathrm{i}k)$ is independent of the τ integration and goes directly into the normalization N_s. Therefore a $\nu = 3/2$ massless propagator is not a two-dimensional h system either, and contributes no new term to Eq. (85), which reads only the h slots of massive propagators. The general half-integer $\nu = \ell + 1/2$ behaves the same way: $z^{\ell+1/2}H^{(1,2)}_{\ell+1/2}(z)$ is a polynomial of degree ℓ times $e^{\pm\mathrm{i}z}$, and term by term reduces to the $\nu = 1/2$ block plus integer powers of τ.

The single-vertex dlog potential of a fixed sector is the full-graph version of Eq. (3.55) of [1]:

$$\boxed{\Omega_{ss} = \Omega_{\mathrm{ex},s} - \mathrm{i}\sum_{v\in V_s} U_s^{-1}\widetilde{\Omega}_{0;v,s}U_s\mathrm{M}_{1;v,s}(\nu_{0,v}+1).} \tag{87}$$

Here the subscripts s, s of Ω_{ss} mark that this diagonal block connects the same sector s, and V_s is the vertex set of that sector. Its letters are exactly all of Eq. (52); no linear solution of the IBP relations is needed first.

The dlog producer and the iterative producer share one registry: the one-dimensional exponential gives $p_{0,v}$, massive blocks give M_1, M_0 and Ω_{ex} at the same time, and massless shared blocks give the two vertices of the propagator the same state bit, opposite incidence and the contact selector. First construct U_s according to Eq. (49), then write $\log\kappa$ state by state, and finally multiply by $\mathrm{M}_1(A+1)$ to obtain Eq. (87). A general inversion of the dense $2^N \times 2^N$ M_0 is never performed here; the actual inversion is only the letter-by-letter reciprocal of the scalar letters of Eq. (51).

If one changes to user coordinates or Wick coordinates, the DE is not rederived; one only pulls back the differential one-form with Eq. (57). If one changes to the package quotient or the physical-derivative basis, one uses the constant similarity transformation (123) or the gauge transformation (109) containing derivative terms, respectively; the three must not be mixed into a single derivative-free basis change.

The off-diagonal blocks of the DE must use the remaining term of the raising relation, i.e. $\mathcal{R}_{v,s}(\boldsymbol{e}_v)$ at $\boldsymbol{n} = \boldsymbol{e}_v$ of Eq. (64) in this handbook (denoted $R^{(1)}$ above); one cannot substitute $\mathcal{R}_{v,s}(\mathbf{0})$ from the lowering step (denoted $R^{(0)}$ above) with its lower time powers reduced first, because the two subsector targets differ by one unit shift and do not commute. The package uses $(-\tau)^A$, whose raising row is

$$\mathrm{M}_{0;v,s}\mathrm{I}_s(\boldsymbol{e}_v) = \mathrm{M}_{1;v,s}(\nu_{0,v}+1)\mathrm{I}_s(0) - \mathcal{R}_{v,s}(\boldsymbol{e}_v). \tag{88}$$

Hence the contact term in the parameter derivative is $+\mathrm{i}\mathrm{M}_0^{-1}\mathcal{R}$; when adapting the τ^ν notation of [1] to this form, this overall sign must not be dropped. If

$$\mathcal{R}_{v,s}(\boldsymbol{e}_v) = \sum_{t\prec s} C^{(v,1)}_{st}\mathrm{I}_t(0), \tag{89}$$

then Eq. (3.68) of [1] generalizes directly to

$$\boxed{\Omega^{\mathrm{contact}}_{st} = +\mathrm{i}\sum_{v\in V_s}\chi_v(E_{st})U_s^{-1}\widetilde{\Omega}_{0;v,s}U_sC^{(v,1)}_{st}, \qquad t\prec s.} \tag{90}$$

Here the subscripts s,t of $\Omega^{\mathrm{contact}}_{st}$ mark the supersector and the lower sector, and the superscript contact marks the contact injection; E_{st} is the set of propagators actually selected by that simultaneous event, and $N_0(E_{st})$ is the number of masslessT propagators in it, with the subscript 0 marking that massless count. One defines

$$\chi_v(E_{st}) = \begin{cases} \varsigma_v, & N_0(E_{st}) = 0, \\ (-1)^{N_0(E_{st})}, & N_0(E_{st}) > 0. \end{cases} \tag{91}$$

Here the subscript v of χ_v marks the vertex into which the contact is injected. For pure-massive events, ς_v is the vertex phase sign fixed at initialization; an event containing massless propagators also includes the phase of the exponential quotient relative to the h/Wronskian contact atoms. All sign-linear correlations are already contained in $C^{(v,1)}_{st}$: the two vertices of a massive propagator use the complementary-bit selector, and the massless quotient uses the $-2,+2$ of Eq. (119). After differentiating the analytic chamber integral of a single pure-massless (anti-)time-ordered propagator with respect to the two vertex energies, Eq. (91) makes the complete DE residual exactly zero component by component; a simultaneous event of three common-theta propagators takes $N_0 = 3$. A pure-massive component after contact must have a single ς; if the input makes the same component mix positive and negative phases, the formula entry fails closed.

9.1 Bit deletion, insertion and the massless quotient in Eq. (3.68) of [1]

First write the pure-massive two-vertex result of Eq. (3.68) of [1], that is, the unreduced form before the extra massless reduction relations are inserted. Let the two states of the pinched propagator sit at positions i,j of the left and right vertices, respectively; the supersector states are $(\boldsymbol{a},\boldsymbol{b})$ and the subsector states are $(\widehat{\boldsymbol{c}},\widehat{\boldsymbol{d}})$, where the hat marks the vector after the pinched position has been deleted. Define

$$K_u^{\mathrm{unreduced}} = \mathrm{T}_{n_u}^{-1}\widetilde{\Omega}_{0;u}\mathrm{T}_{n_u}, \qquad K_v^{\mathrm{unreduced}} = \mathrm{T}_{n_v}^{-1}\widetilde{\Omega}_{0;v}\mathrm{T}_{n_v}. \tag{92}$$

Here the subscripts of $K_u^{\mathrm{unreduced}}$ and $K_v^{\mathrm{unreduced}}$ mark the endpoint vertices, and the superscript unreduced marks that the extra massless relations have not yet been inserted; T_{n_u} is the tensor product $\mathrm{T}^{\otimes n_u}$ acting on the n_u two-dimensional factors of that vertex. Then the two terms of that equation, one at each of the two vertices, read component by component

$$\begin{aligned} &\left(\Omega^{\mathrm{unreduced}}_{s,s/e}\right)_{(\boldsymbol{a},\boldsymbol{b}),(\widehat{\boldsymbol{c}},\widehat{\boldsymbol{d}})} \\ &\quad = -\mathrm{i}(K_u^{\mathrm{unreduced}})_{\boldsymbol{a},\mathrm{Ins}_i(\widehat{\boldsymbol{c}},1-b_j)}(-1)^{b_j}\delta_{\mathrm{Del}_j\,\boldsymbol{b},\widehat{\boldsymbol{d}}} \\ &\qquad - \mathrm{i}(K_v^{\mathrm{unreduced}})_{\boldsymbol{b},\mathrm{Ins}_j(\widehat{\boldsymbol{d}},1-a_i)}(-1)^{a_i}\delta_{\mathrm{Del}_i\,\boldsymbol{a},\widehat{\boldsymbol{c}}}. \end{aligned} \tag{93}$$

This is the precise meaning of "deleting the bit of the pinched propagator at one vertex and inserting its complementary bit on the other side"; all undeleted bits are preserved by the Kronecker deltas. The $(-\tau)^A$ normalized-J convention of this handbook converts the overall $-\mathrm{i}$ of Eq. (93) into the $+\mathrm{i}\chi_v(E)$ of Eq. (90) according to Eq. (88); the fixed positive pure-massive vertices of [1] take $\chi_v = \varsigma_v = +1$. One must not change only the binary selector and miss this convention transformation.

Now pinch a massless propagator e. Since the shared slot breaks the vertex-by-vertex factorization, first define the global kernel on the full supersector

$$\mathcal{K}_{v,s} = U_s^{-1}\widetilde{\Omega}_{0;v,s}U_s. \tag{94}$$

where the subscript v marks the vertex and s the sector. In the supersector global binary string after the quotient, the propagator has a single shared bit, whose position is denoted by p_e; the subsector string is denoted by $\widehat{\boldsymbol{c}}$. The contact matrix components in the package basis are

$$\left(C^{(v,e)}_{s,s/e}\right)_{\boldsymbol{d},\widehat{\boldsymbol{c}}} = \eta_{s,e}\, w_e^{(v)}\delta_{d_{p_e},1}\delta_{\mathrm{Del}_{p_e}\,\boldsymbol{d},\widehat{\boldsymbol{c}}}, \tag{95}$$

where $\eta_{s,e}$ collects the already-absorbed sector-normalization and pinch prefactor, with $w_e^{(u_e)} = -2$ and $w_e^{(v_e)} = +2$. The intermediate-state sum in Eq. (90) then collapses directly:

$$\sum_{\boldsymbol{d}} (\mathcal{K}_{v,s})_{\boldsymbol{a},\boldsymbol{d}} \delta_{d_{p_e},1} \delta_{\mathrm{Del}_{p_e}\boldsymbol{d},\widehat{\boldsymbol{c}}} = (\mathcal{K}_{v,s})_{\boldsymbol{a},\mathrm{Ins}_{p_e}(\widehat{\boldsymbol{c}},1)}. \tag{96}$$

The complete top-to-sub block of a single massless propagator is therefore

$$\boxed{\left(\Omega^{(e)}_{s,s/e}\right)_{\boldsymbol{a},\widehat{\boldsymbol{c}}} = -\mathrm{i}\eta_{s,e}\left[w_e^{(u_e)}(\mathcal{K}_{u_e,s})_{\boldsymbol{a},\mathrm{Ins}_{p_e}(\widehat{\boldsymbol{c}},1)} + w_e^{(v_e)}(\mathcal{K}_{v_e,s})_{\boldsymbol{a},\mathrm{Ins}_{p_e}(\widehat{\boldsymbol{c}},1)}\right]}$$
$$= 2\mathrm{i}\eta_{s,e}\left[(\mathcal{K}_{u_e,s})_{\boldsymbol{a},\mathrm{Ins}_{p_e}(\widehat{\boldsymbol{c}},1)} - (\mathcal{K}_{v_e,s})_{\boldsymbol{a},\mathrm{Ins}_{p_e}(\widehat{\boldsymbol{c}},1)}\right]. \tag{97}$$

Thus one does not keep two independent 2×2 vertex indices and then sum: the $4 \to 2$ quotient has already turned them into one shared bit; the so-called "sum over the two vertices of the propagator" is only the addition of the two component log kernels in Eq. (97).

If another massless propagator $f \neq e$ is not pinched, its shared bit is a spectator and automatically gives

$$\delta_{r_f^{\mathrm{sup}},r_f^{\mathrm{sub}}}. \tag{98}$$

in Eq. (95). It does not produce an extra sum over the bits at its two vertices; it only continues to enter the energy letters and $\mathcal{K}_{v,s}$ of its two vertices with opposite incidence signs. In the mixed massive/massless case, one only needs to assemble the bits of the massive propagators and the other shared bits into the same global string according to Eq. (60); after deleting p_e, all remaining bits are preserved by the second delta of Eq. (95).

If a generalized or multi-propagator contact makes $R^{(1)}$ land on a nonzero shift $\boldsymbol{q}$ of the subsector, the program now calls the same recurrence formula subsector state by subsector state and constructs

$$\mathrm{I}_t(\boldsymbol{q}) = \mathcal{Q}_t(\boldsymbol{q} \to 0)\,\mathrm{I}_{\mathrm{all}}(0), \qquad \Omega^{(v)}_{s\to *} = -\mathcal{K}_{v,s} C^{(v,1)}_{st}\,\mathcal{Q}_t(\boldsymbol{q}_v \to 0). \tag{99}$$

Here $\mathrm{I}_{\mathrm{all}}(0)$ is ordered according to the global master-integral order of the context, so $\mathcal{Q}_t$ is allowed to continue down to the lower contact sectors of t, rather than keeping only the direct subsector. The program stores, column by column, the residual, the unresolved shifted integrals and the direction-sensitive singular layers. Only when all columns close is Eq. (99) written into the connection; otherwise it returns

`contactShiftReductionFailed`.

An ordinary single-propagator contact still gives $\boldsymbol{q} = 0$ directly by the base-power choice.

After ordering the sectors by the number of remaining (anti-)time-ordered propagators, the complete connection is block triangular:

$$\mathrm{d}\begin{pmatrix} \mathrm{I}_{s_0} \\ \mathrm{I}_{s_1} \\ \vdots \end{pmatrix} = \begin{pmatrix} \Omega_{s_0 s_0} & \Omega_{s_0 s_1} & \cdots \\ 0 & \Omega_{s_1 s_1} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \begin{pmatrix} \mathrm{I}_{s_0} \\ \mathrm{I}_{s_1} \\ \vdots \end{pmatrix}. \tag{100}$$

Whether the matrix is written as upper or lower triangular depends only on the sector ordering; the physical content is that contact only points from a supersector to a strictly lower sector.

10 The dimensionless h equation, physical quadratic-momentum terms and realification

For the $\nu = 1/2$ massless building block, Eq. (12) is the equation to be used inside the block:

$$\partial_z^2 h_0 = -h_0. \tag{101}$$

Restoring the physical time derivative within that block with Eq. (10) gives

$$\boxed{\partial_\tau^2 h_0 = k^2 \partial_z^2 h_0 = -k^2 h_0.} \tag{102}$$

So $-k^2$ is not a second set of EOMs; it is the physical coefficient of the dimensionless constant -1 after two chain-rule steps. Continuing to use the two states $h_0 = h(\nu, 0; z)$ and $h_1 = h(\nu, 1; z)$ internally allows the 2×2 formulas of [1] to be reused unchanged.

If the backend realification uses

$$k = -\mathrm{i}\kappa, \tag{103}$$

then

$$-k^2 = +\kappa^2. \tag{104}$$

If the program token for the real variable happens to be named `ik`, the `ik^2` in the output means the square of this real variable, not the imaginary unit i times k^2.

Now distinguish the h-state convention of [1], called the reference h-state below, from the physical derivatives at the vertices. The integrals of Sec. 6 keep the symbol I_{ab} there and are dimensionless; the physical-derivative integrals defined now are written $\mathcal{F}_{ab}$, and the two differ by the powers of $-k$ collected in the matrix $N(k)$ below. Let

$$\mathcal{F}_{ab} = \partial_{\tau_u}^a \partial_{\tau_v}^b G_e \quad (a, b = 0, 1) \tag{105}$$

denote the physical states from which $-k$ has not been divided out; the subscripts a, b are the numbers of derivatives at the first and second endpoints u_e, v_e, respectively. The single-vertex top-sector part satisfies

$$\mathcal{F}_{01} = -\mathcal{F}_{10}, \qquad \mathcal{F}_{11} = k^2 \mathcal{F}_{00}, \tag{106}$$

while the second relation of the complete (anti-)time-ordered propagator must additionally receive the contact distribution from the theta derivative, which is exactly the lower sector to which Eq. (78) belongs.

If one chooses the physical two-state $\boldsymbol{g}_{\mathrm{phys}} = (\mathcal{F}_{00}, \mathcal{F}_{10})^T$, its relation to the reference h two-state is

$$\boldsymbol{g}_{\mathrm{phys}} = N(k) \boldsymbol{g}_e, \qquad N(k) = \begin{pmatrix} 1 & 0 \\ 0 & -k \end{pmatrix}. \tag{107}$$

The single-vertex embedding of the physical four states is therefore

$$\begin{pmatrix} \mathcal{F}_{00} \\ \mathcal{F}_{01} \\ \mathcal{F}_{10} \\ \mathcal{F}_{11} \end{pmatrix}_{\mathrm{reg}} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ k^2 & 0 \end{pmatrix} \boldsymbol{g}_{\mathrm{phys}}. \tag{108}$$

Here the subscript reg denotes the regular part: only the terms from ordinary derivatives of the propagator are kept, excluding the contact distributions produced by theta derivatives. Hence if the final DE uses the physical two-state rather than the reference h two-state, one must perform

$$\boxed{\mathcal{A}_{\mathrm{phys}} = \mathrm{d}N\, N^{-1} + N \mathcal{A}_h N^{-1}, \qquad \mathrm{d}N\, N^{-1} = \begin{pmatrix} 0 & 0 \\ 0 & \mathrm{d}\log k \end{pmatrix}.} \tag{109}$$

where $\mathcal{A}_h$ and $\mathcal{A}_{\mathrm{phys}}$ are the DE matrices in the h-state and physical-derivative bases, respectively, and $N(k)$ is the basis-change matrix of Eq. (107). It is recommended that the formula producer always keep the dimensionless h-state basis internally, so that S_e, P_e, T, U_s are all kinematics-independent constants; only when outputting the physical-derivative states should $N(k)$ and Eq. (109) be applied.

11 Interface with the package conventions

11.1 The massless quotient basis of the package

The I_{ab} above are the reference h-states. In order to separate the momentum factor of the derivative at each vertex from the discrete states, the package uses another algebraic quotient. For an ordered propagator $e = (u_e, v_e)$, let $\Delta = \tau_{u_e} - \tau_{v_e}$, and use $\sigma_e = +1, -1$ for G_{++}, G_{--} respectively; define

$$\mathsf{m}_{e,0} = \theta(\Delta) e^{-\mathrm{i}\sigma_e q_e \Delta} + \theta(-\Delta) e^{+\mathrm{i}\sigma_e q_e \Delta}, \tag{110}$$

$$\mathsf{m}_{e,1} = -\theta(\Delta) e^{-\mathrm{i}\sigma_e q_e \Delta} + \theta(-\Delta) e^{+\mathrm{i}\sigma_e q_e \Delta}. \tag{111}$$

Here $\mathsf{m}_{e,0}, \mathsf{m}_{e,1}$ are the two quotient states of propagator e: the subscript e marks the propagator and the second subscript $0, 1$ marks the state bit. The shared two-state and the four-state embedding of the package are

$$\boldsymbol{g}_e^J = (\mathsf{m}_{e,0}, \mathsf{m}_{e,1})^T, \qquad \begin{pmatrix} \mathrm{I}_{00}^J \\ \mathrm{I}_{01}^J \\ \mathrm{I}_{10}^J \\ \mathrm{I}_{11}^J \end{pmatrix} = S_e^J \boldsymbol{g}_e^J, \qquad S_e^J = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ -1 & 0 \end{pmatrix}. \tag{112}$$

Here the superscript J marks the normalized-J (quotient) basis, and S_e^J is the embedding in that basis. So in the package one indeed has

$$\mathrm{I}_{01}^J = -\mathrm{I}_{10}^J, \qquad \mathrm{I}_{11}^J = -\mathrm{I}_{00}^J. \tag{113}$$

This minus sign does not conflict with Eq. (28): I_{ab}^J has already factored out the $\pm\mathrm{i}\sigma_e q_e$ of the derivatives at the two vertices, so it is not $h_a h_b$. Under a common kinematics-independent normalization, the two shared two-states satisfy

$$\boldsymbol{g}_e^J = D_{\sigma_e} \boldsymbol{g}_e, \qquad D_{\sigma_e} = \mathrm{diag}(1, \mathrm{i}\sigma_e). \tag{114}$$

Therefore the $\mp\mathrm{i}q_e\sigma_2$ of the two vertices of the propagator in the reference h-state become, in the package quotient, exactly

$$Q_{u_e,e}^J = D_{\sigma_e}(-\mathrm{i}q_e\sigma_2) D_{\sigma_e}^{-1} = +\mathrm{i}\sigma_e q_e \sigma_1, \tag{115}$$

$$Q_{v_e,e}^J = D_{\sigma_e}(+\mathrm{i}q_e\sigma_2) D_{\sigma_e}^{-1} = -\mathrm{i}\sigma_e q_e \sigma_1, \qquad \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}. \tag{116}$$

The equivalent complete rules at the two vertices are

$$\partial_{\tau_{u_e}} \mathsf{m}_{e,n} = +\mathrm{i}\sigma_e q_e \mathsf{m}_{e,1-n} - 2n\,\delta(\Delta), \tag{117}$$

$$\partial_{\tau_{v_e}} \mathsf{m}_{e,n} = -\mathrm{i}\sigma_e q_e \mathsf{m}_{e,1-n} + 2n\,\delta(\Delta). \tag{118}$$

This simultaneously gives the contact weights of the two vertices u_e and v_e, in that order, in the package quotient

$$w_{e,J}^{(u_e)} = -2, \qquad w_{e,J}^{(v_e)} = +2. \tag{119}$$

They agree exactly with Eq. (37), because

$$D_{\sigma_e}|1\rangle_e\,(+2\mathrm{i}\sigma_e) = -2|1\rangle_e, \qquad D_{\sigma_e}|1\rangle_e\,(-2\mathrm{i}\sigma_e) = +2|1\rangle_e. \tag{120}$$

A cycle propagator stores each factor of q_e as $b_e \to b_e - 1$; a fixed/non-loop propagator has no b_e slot and must keep the structured modulus q_e explicitly. Two single-vertex derivatives at the same vertex therefore give $-q_e^2$, while one at each of the two vertices gives $+q_e^2$.

The diagonalizing matrix of the package quotient can be obtained directly from the matrix of [1]:

$$\mathrm{T}_e^J = \mathrm{T} D_{\sigma_e}^{-1}, \qquad \mathrm{T}_e^J Q_{u_e,e}^J (\mathrm{T}_e^J)^{-1} = \mathrm{T}(-\mathrm{i} q_e \sigma_2)\mathrm{T}^{-1}. \tag{121}$$

So the two bases give the same set of energy letters; σ_e only swaps the label of the shared bit. If one instead uses the fixed real Hadamard matrix, this swap appears explicitly as $\sigma_e(2r_e - 1)q_e$ in the letter.

For the whole sector, embed Eq. (114) into the shared slot of every unpinched massless (anti-)time-ordered propagator and denote the resulting constant matrix by

$$B_s = \bigotimes_{e \in E_m(s)} D_{\sigma_e}, \tag{122}$$

with identity matrices on the remaining slots. All single-vertex and contact blocks can then be transformed directly as

$$\mathrm{M}_{0;v,s}^J = B_s \mathrm{M}_{0;v,s}^h B_s^{-1}, \qquad \Omega_{ss}^J = B_s \Omega_{ss}^h B_s^{-1}, \qquad C_{st}^J = B_s C_{st}^h B_t^{-1}. \tag{123}$$

The last equation is a basis change between spaces of different dimensions, not a similarity transformation; when $s \to t$ deletes the propagator e, the left multiplication by D_{σ_e} exactly accomplishes Eq. (120).

11.2 The unique quotient and its four-state derivation space

MadStree uniquely uses one shared two-state `masslessShared` slot for every `masslessT`. The public input has no representation-selection option; recurrence, contact, dlog and numerical layers consume only the quotient. The four-state space of the two ordered vertices $(00, 01, 10, 11)$ below is used only to derive that quotient and is not a second program context, slot registry or formula route.

For the derivation, work first in a four-state space with the constant normalization compatible with the quotient, without introducing the $2/\pi$ of the bare Hankel product. Using S_e, P_e of Eq. (30), define

$$S_{h \leftarrow J} = S_e D_\sigma^{-1}, \qquad P_{J \leftarrow h} = D_\sigma P_e, \qquad P_{J \leftarrow h} S_{h \leftarrow J} = \mathbb{I}_2. \tag{124}$$

For G_{++} this gives

$$S_{h \leftarrow J} = \begin{pmatrix} 1 & 0 \\ 0 & \mathrm{i} \\ 0 & -\mathrm{i} \\ 1 & 0 \end{pmatrix}, \qquad P_{J \leftarrow h} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & \mathrm{i} & 0 \end{pmatrix}. \tag{125}$$

The contact column of the first of the two vertices in the four-state derivation space is $2\mathrm{i}\sigma(0, -1, 1, 0)^T$, and the second vertex takes the opposite sign; after projecting with $P_{J \leftarrow h}$, one obtains exactly the $-2, +2$ of Eq. (119). For a sector with spectator slots, Eq. (124) shows how the four-state space is projected once into the shared production two-state space. The program then generates $\mathrm{M}_1, \mathrm{M}_0$, the iterative reduction and the complete block-triangular dlog only in the quotient space and retains no four-state production branch.

11.3 $(-\tau)^A$ and normalized-J

The previous text used the τ^{ν_0} convention of [1]. The integral object used by recurrence, DEs and numerical transport is the normalized object

$$J_s = \mathcal{N}_s \mathrm{I}_s, \tag{126}$$

where the bare integral with the same existing indices is defined compactly by

$$\mathrm{I}_s(\boldsymbol{n};\boldsymbol{a}) = \int_{(-\infty,0)^{c_s}} \prod_{v\in V_s} \mathrm{d}\tau_v\, (-\tau_v)^{\nu_{0,v}+n_v} e^{\mathrm{i}p_{0,v}\tau_v}\; \mathcal{H}_{s,\boldsymbol{a}}(\boldsymbol{\tau}). \tag{127}$$

Here $s =$ `sectorKey`, $\boldsymbol{n} =$ `aShifts` and $\boldsymbol{a} =$ `stateBits`; $\mathcal{H}_{s,\boldsymbol{a}}$ is the ordered product of the two-state building blocks of the active propagators defined above. Massive propagators read their own state bits, whereas a massless shared quotient reads one bit shared by the whole propagator. The $++/--$ time ordering of an (anti-)time-ordered propagator stays inside its building block, and pinched propagators are absent. The program does not expand this product separately for every master.

The single-integral query is

```
MSIntegralDefinition[MSIntegral[s,n,a], context]
```

It returns the same-index `MSBareIntegral[s,n,a]`, the exact $\mathcal{N}_s$, and an inert equality; each ordered record returned by `MSMasterIntegrals[context]` contains the same fields. For $\mathcal{N}_s = 1$ the displayed relation is $J_s(\boldsymbol{n};\boldsymbol{a}) = \mathrm{I}_s(\boldsymbol{n};\boldsymbol{a})$. The time power is written as $(-\tau_v)^{\nu_{0,v}+n_v}$, so

$$\partial_{\tau_v}(-\tau_v)^{\nu_{0,v}+n_v} = -(\nu_{0,v}+n_v)(-\tau_v)^{\nu_{0,v}+n_v-1}. \tag{128}$$

In the same h-state basis, the package adapter therefore writes Eq. (66) of this handbook exactly as

$$-\mathrm{M}_{1;v,s}(\nu_{0,v}+n_v)\boldsymbol{J}_s(\boldsymbol{n}-\boldsymbol{e}_v) + \mathrm{M}_{0;v,s}\boldsymbol{J}_s(\boldsymbol{n}) + \mathcal{R}^J_{v,s}(\boldsymbol{n}) = 0, \tag{129}$$

where the minus sign comes entirely from Eq. (128). If the package quotient basis is used, one only needs to apply the constant similarity transformation of Eq. (114) to M_0 on each massless shared slot; M_1 does not contain that slot. This adaptation does not change the shared-slot registry.

Under the package conventions the two one-step formulas are therefore explicitly

$$\boldsymbol{J}_s(\boldsymbol{n}-\boldsymbol{e}_v) = \mathrm{M}_{1;v,s}(\nu_{0,v}+n_v)^{-1}\left[\mathrm{M}_{0;v,s}\boldsymbol{J}_s(\boldsymbol{n}) + \mathcal{R}^J_{v,s}(\boldsymbol{n})\right], \tag{130}$$

$$\boldsymbol{J}_s(\boldsymbol{n}+\boldsymbol{e}_v) = \mathrm{M}^{-1}_{0;v,s}\left[\mathrm{M}_{1;v,s}(\nu_{0,v}+n_v+1)\boldsymbol{J}_s(\boldsymbol{n}) - \mathcal{R}^J_{v,s}(\boldsymbol{n}+\boldsymbol{e}_v)\right]. \tag{131}$$

This preserves the directionality of Eqs. (76)–(77); the overall sign change does not alter which matrix is inverted in that direction.

In the normalized basis one must also use

$$\partial_x J_s = (\partial_x \log\mathcal{N}_s)J_s + \mathcal{N}_s\partial_x \mathrm{I}_s, \qquad C^J_{st} = \frac{\mathcal{N}_s}{\mathcal{N}_t}C^I_{st}. \tag{132}$$

So the sector diagonal blocks receive an additional $\mathrm{d}\log\mathcal{N}_s\mathrm{I}_s$, and the off-diagonal contact blocks are multiplied by $\mathcal{N}_s/\mathcal{N}_t$. Only when this ratio has been turned into a kinematics-independent constant by the basis normalization is Eq. (90) directly a constant-residue dlog; otherwise an additional basis/gauge transformation is needed, and one cannot claim completion merely from the literal $\log\kappa$.

The public time-only representation of dSIBP is `J[sectorKey,timeShifts,stateBits]`, which corresponds slot by slot to MadStree's `MSIntegral[sectorKey,aShifts,stateBits]`; `stateBits` are discrete states of the n_i type in the full-sector registry and are not required to be grouped by vertex. This correspondence only converts the integral identity and does not replace the normalization: both contexts must use the same $\mathcal{N}_s$ for every sector, and differentiation must include the $\partial\log\mathcal{N}_s$ of Eq. (132). The default massless input of MadStree is $\nu = 1/2$ with positive prefactor; in this case `formulaNu`$= s_\nu\nu = +1/2$, and the code and the formulas of this handbook agree slot by slot on the same $+\nu$ base formula. With `NuConvention -> "Negative"` one has `formulaNu`$= -1/2 = \nu_{\mathrm{ref}}$, used only to compare verbatim with the printed formulas of that paper. This convention does not replace propagator metadata, theta, or contact normalization.

11.4 Single-vertex families, local inverses and the complete reduction interface

A single-vertex family does not need to construct a fake graph. The compact input is

```
ctx = MSInitVertexFamily[<|
  "ki" -> {k0,k1,...},
  "nui" -> {A0,nu1,...},
  "hankelBranches" -> {1,2,...}
|>, NuConvention -> "Positive"];
```

Here `First[ki]` and `First[nui]` are respectively the parameter of the theta-free exponential attached to the vertex and the base power of $(-\tau)$; the remaining entries give the momentum, the ν used as-is as the Hankel order, and the Hankel branch of each h block. The explicit data model uses `externalLegEnergy/timePower/hBlocks/exponentialBlocks`. `exponentialBlocks` is only a convenience of the input layer: at initialization its entries are folded into the effective external-leg energy with their literal `exponentType` signs, after which only the single parameter of Eq. (5) remains and no independent slot is created, while each h block is a two-dimensional slot of a massive propagator. The context can directly call `MSFormulaMatrices`, `MSMasterIntegrals`, `MSDLogDE`, and `MSBoundaryData`, then use `MSEvaluatePath` for the one-stage numerical workflow. `NuConvention` is chosen only at initialization and all later interfaces read it from the context.

The complete explicit model has the shape

```
ctx = MSInitVertexFamily[<|
  "externalLegEnergy" -> k0,
  "timePower" -> A0,
  "hBlocks" -> {
    <|"id"->h1,"momentum"->k1,"nu"->nu1,"hankelBranch"->1|>
  },
  "exponentialBlocks" -> {
    <|"id"->e1,"momentum"->p1,"exponentType"->"-"|>
  },
  "vertexType" -> "+",
  "normalization" -> 1
|>];
```

Here `vertexType` fixes the SK contour branch of the whole vertex, while `exponentType` is only the literal sign of that extra exponential block in the effective external-leg energy. A single-vertex family has no propagator incidence, so its initialization schema intentionally differs from tree `vertices/lines`. Once the context exists, both forms use the same masters, recurrence, DE, boundary, numerical options, and return structure.

The implementation contains no general $2^p \times 2^p$ symbolic inverse. For a massive propagator, the local non-trivial direction of M_0 is σ_2, and one directly uses the constant matrix T of [1] with its explicit inverse; for the shared slot of a massless propagator, the local direction is σ_1, and one uses the self-inverse Hadamard matrix

$$H_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad H_2^{-1} = H_2. \tag{133}$$

Pure exponential factors are one-dimensional identity matrices. The full-sector U_s and U_s^{-1} are obtained only as Kronecker products of these local matrices. Since M_1 is already diagonal in the state-bit basis, `msM1Inverse` only computes

$$\mathrm{diag}(\mathrm{M}_1^{-1})_r = \lambda_{1,r}^{-1}; \tag{134}$$

while M_0 is computed only in the common diagonal basis as

$$\mathrm{M}_0^{-1} = U_s^{-1}\,\mathrm{diag}\left((\mathrm{i}\kappa_r)^{-1}\right) U_s. \tag{135}$$

Thus after the massless $4 \to 2$ quotient the local-diagonalization inversion still applies unchanged. The necessary condition is that all local operators on the same slot contain only the same Pauli direction; if non-collinear directions appear in the future, the program must reject Eq. (135) instead of pretending they can be simultaneously diagonalized.

The complete reduction entry point is

```
MSReduce[linearCombination, context, MasterBasis -> Automatic]
```

It recursively moves each component shift to zero for every `MSIntegral` according to Eqs. (130)–(131), and continues to reduce the lower sectors along the contact DAG; repeated subproblems are reused through a memo table. The key returned fields are

```
result, masterBasis, masterRules, coefficientVector,
nonMasterResidual, remainingShiftedIntegrals,
memoizedIntegralCount, singularLayers
```

where the master basis, the coefficient vector and the rules are strictly in the same order, and `result` is the explicit linear combination of master integrals. The status is `reduced` only when the residual is zero and the shifted-integral list is empty. `MasterBasis` may be any permutation of all context masters, but it cannot miss or repeat entries. The singular layers record, for each step, the direction, the component and the denominator that was actually inverted: toward zero with a positive shift only the energy letters of M_0 are reported; with a negative shift only the M_1 eigenvalues are reported. This is exactly the program contract of Eqs. (76)–(77). The reduction runs on the vertex time powers fixed at initialization: to substitute numerical or regulated powers (such as $a_1 = -1 + \epsilon$), re-initialize the context with these values as `timePower` and reduce there (Example 06 demonstrates both paths), or substitute into the symbolic result afterwards; `MSReduce` itself carries no substitution option.

The same entry directly accepts `List`, regular `Table`, empty lists and ragged nested lists. The returned `result`, per-element `coefficientVector/masterRules`, residuals and remaining shifts preserve the input shape; all elements share one recursion memo. A failed element reports its exact list position and does not mark the partial results of other elements as a complete reduction.

11.5 The small-time end of the integration path and the safe time-IBP shift

Integer time-IBP shifts may not cross a divergent point on the path. MadStree does not solve a new local series here. It substitutes only the known two leading small-time powers of h into every reachable component and enumerates all binary branch combinations of that component. For a zero-shift Gamma argument α, if $\alpha \in \{0, -1, -2, \ldots\}$, the original master diverges and initialization is rejected immediately. If $\alpha = m \in \mathbb{Z}_{>0}$, initialization is allowed, but that branch is restricted to

$$n \geq 1 - m, \tag{136}$$

because the next shift reaches a non-positive integer. `MSRecurrenceStep`, `MSReduce`, and shifted-subsector reduction in the dlog reject only an actual request that crosses this bound, returning the original vertices, sector, local branch, α, and the requested shift. An α containing a symbolic analytic regulator remains symbolic and is not misclassified as an integer.

12 Pure-exponential-energy infinity boundary and blow-up for arbitrary time-only graphs

12.1 Damping variables and auxiliary one-dimensional exponentials

For each root vertex v, choose the pure vertex exponential of Eq. (53) and use the internal positive variable K_v of Eq. (55). Since $\tau_v = -t_v < 0$, both kinds of vertices satisfy

$$\exp(\mathrm{i}\varsigma_v k_{0,v}\tau_v) = \exp(-K_v t_v), \qquad K_v > 0. \tag{137}$$

So $K_v \to \infty$ localizes the integral to $\tau_v \to 0^-$. This is the same as the single-vertex family boundary of Sec. 3 of [2]; that reference uses the chart with $\varsigma_v = +1$ and $k_{0,v} \to -\mathrm{i}\infty$.

If the original integrand has no pure vertex exponential at some vertex, one may add an auxiliary energy $\exp(\mathrm{i}\varsigma_v k_{0,v}^{\mathrm{aux}} \tau_v)$. It only maps

$$p_{0,v} \longmapsto p_{0,v} + \varsigma_v k_{0,v}^{\mathrm{aux}}, \qquad \mathrm{M}_{0;v,s} \longmapsto \mathrm{M}_{0;v,s} + \mathrm{i}\varsigma_v k_{0,v}^{\mathrm{aux}} \mathbb{I}_s. \tag{138}$$

Hence under generic kinematics it does not increase the number of slots or masters and does not change M_1. However, $K_v^{\mathrm{aux}} = 0$ is a specialization of this augmented family; whether the point is still ordinary must be decided by the full-sector letters and cannot be inferred from "a one-dimensional exponential does not increase the masters."

12.2 Strict total time order, nested blow-up and theta fixing

Directly choose a strict total order for all root times, and take the opposite strict order of damping energies; here σ is a permutation of the vertices and c_V is the number of vertices:

$$K_{\sigma(1)} \gg K_{\sigma(2)} \gg \cdots \gg K_{\sigma(c_V)} \gg 1. \tag{139}$$

Here $\mathrm{rank}(u)$ (also written r_u) is the position of $u = \sigma(j)$ in this strict total order, with $j = 1$ corresponding to the largest damping energy; r_C below is the largest rank inside the component. This step does not require the propagator incidence graph to be a tree. What this handbook calls a time-only loop graph is a graph that is allowed to contain loops, but where all propagator momenta are still treated as fixed parameters and only the time integrals of the vertices are performed; it does not include loop-momentum integration. A strict total order orders every pair of times, and for non-adjacent vertices the order merely selects one blow-up chart.

Define the nested coordinates

$$x_j = \frac{K_{\sigma(j+1)}}{K_{\sigma(j)}} \quad (1 \le j < c_V), \qquad x_{c_V} = \frac{1}{K_{\sigma(c_V)}}. \tag{140}$$

where the subscript j of x_j marks the nesting level, with c_V coordinates in total. Then

$$\frac{1}{K_{\sigma(j)}} = \prod_{r=j}^{c_V} x_r, \qquad x_1, \ldots, x_{c_V} \longrightarrow 0. \tag{141}$$

For two vertices with $K_1 \gg K_2$, this is the $x = K_2/K_1$, $y = 1/K_2$ of Sec. 4.1 of the reference. In the original inverse variables, the three divisors $t_1, t_2, t_1 + t_2$ pass through the two-dimensional origin simultaneously, a degenerate intersection; Eq. (140) resolves it analytically into a normal-crossing chart.

The exponential localization scales satisfy

$$|\tau_{\sigma(1)}| \ll |\tau_{\sigma(2)}| \ll \cdots \ll |\tau_{\sigma(c_V)}| \ll 1. \tag{142}$$

Since all τ approach zero from the negative side, large K corresponds to small $|\tau|$; hence $K_u \gg K_v$ implies $\tau_u > \tau_v$, and therefore

$$\theta(\tau_u - \tau_v) \longrightarrow 1, \qquad \theta(\tau_v - \tau_u) \longrightarrow 0. \tag{143}$$

For G_{++} or G_{--}, Eq. (143) must be applied to every explicit theta in the propagator expansion according to the order of its argument; one cannot guess a single value from the SK label alone. All thetas in an arbitrary time graph therefore become 0 or 1. If some term requires a cyclic inequality, for example

$$\theta(\tau_1 - \tau_2)\theta(\tau_2 - \tau_3)\theta(\tau_3 - \tau_1), \tag{144}$$

then it is automatically zero in every strict-total-order chart, instead of producing a new boundary region.

12.3 Choosing the preferred rank from a user-preferred ordinary point

The user may additionally provide a preferred starting point P_*. The point is still submitted in the user variables of initialization, including all $k_{0,v}$ and all propagator moduli q_e; the package first computes with the coordinate map of Eq. (57)

$$K_v(P_*) = \mathrm{i}\varsigma_v k_{0,v}(P_*). \tag{145}$$

If $\operatorname{Re} K_v(P_*) > 0$, the preferred strict chart takes

$$\operatorname{Re} K_{\sigma(1)}(P_*) > \cdots > \operatorname{Re} K_{\sigma(c_V)}(P_*), \tag{146}$$

with ties broken deterministically first by $|K_v(P_*)|$ and then by the root vertex order. The order is not reversible: a large K_v supports only a smaller typical $t_v = |\tau_v|$ to avoid the exponential suppression of $e^{-K_v t_v}$, so Eq. (146) corresponds to

$$|\tau_{\sigma(1)}| \ll \cdots \ll |\tau_{\sigma(c_V)}|, \qquad \tau_{\sigma(1)} > \cdots > \tau_{\sigma(c_V)}. \tag{147}$$

This choice makes the scale hierarchy of the boundary chart closest to P_*, usually shortening the first continuation segment and possibly reducing the letter hypersurfaces that must be bypassed; it is not a theorem of "globally fewest singularities." The program must still issue a normal-crossing certificate for the complete top/subsector connection of the candidate chart, and verify that the path from an ordinary anchor inside the chart to P_* does not cross any letter. If some $\operatorname{Re} K_v(P_*) \le 0$, that point cannot directly define a damping rank; it may still be used as a later ordinary point, but the preferred chart is then given explicitly by the user or determined by another positive-damping anchor.

12.4 Contact subsectors inherit the same root rank

In a sector s, the delta contact merges a set of root vertices into a component C. Its damping energy and rank are defined as

$$K_C = \sum_{u\in C} K_u, \qquad r_C = \max_{u\in C} r_u. \tag{148}$$

The strict total order guarantees that K_C is dominated by the unique largest-rank root in C. The delta merge changes the component base time power and the sector normalization, but does not reselect a subsector rank; when a remaining theta connects two components, only their dominant roots are compared.

Let v_C be the dominant root in C. The boundary form of any letter of Eq. (52) is

$$\begin{aligned} \kappa_{C,s;\boldsymbol{a},\boldsymbol{r}} &= -\mathrm{i}K_C + E_{C,s;\boldsymbol{a},\boldsymbol{r}} \\ &= -\mathrm{i}K_{v_C}\left[1 + \text{higher monomials in } x + \frac{\mathrm{i}E_{C,s;\boldsymbol{a},\boldsymbol{r}}}{K_{v_C}}\right], \end{aligned} \tag{149}$$

where the bracket equals 1 at $x = 0$. All energy letters of every contact-reachable sector are therefore a coordinate monomial times a boundary-nonzero unit; this is the direct resolution of the linear letters of arbitrary time graphs by the strict-rank blow-up.

This conclusion is in fact more general than the component letters. The diagonal blocks of a fixed sector contain only the $\log\kappa$ of Eq. (84). The top-to-sub formula (90) merely multiplies the same $\widetilde{\Omega}_{0;v,s}$ of the supersector by the kinematics-independent contact map $C_{st}^{(v,1)}$, so no new log argument is produced. If $R^{(1)}$ does not land directly on the subsector master and further reduction is needed, the new denominators come only from the M_0^{-1} of the subsector and are still energy letters of the same type of the subsector component. Inducting along the strictly

descending contact DAG, every denominator containing K in the complete block-triangular DE has the form

$$D_{\alpha,s}(K,q) = Q_{\alpha,s}(q) + \sum_{v=1}^{c_V} c_{\alpha,s;v} K_v, \qquad \operatorname{supp} c_{\alpha,s} \neq \varnothing, \tag{150}$$

where $c_{\alpha,s;v}$ is kinematics-independent. For an ordinary component letter, c_v is exactly the indicator of the component; more general constant contact bases or SK phases only change the nonzero constant coefficients.

Let

$$j_\alpha = \min\{j \mid c_{\alpha,s;\sigma(j)} \neq 0\} \tag{151}$$

be the first position of the support of this denominator in the strict energy total order. Substituting Eq. (140), it factorizes strictly as

$$\begin{aligned} D_{\alpha,s} = K_{\sigma(j_\alpha)} \Bigg[& c_{\alpha,s;\sigma(j_\alpha)} + \sum_{j>j_\alpha} c_{\alpha,s;\sigma(j)} \prod_{r=j_\alpha}^{j-1} x_r \\ & + Q_{\alpha,s} \prod_{r=j_\alpha}^{c_V} x_r \Bigg]. \end{aligned} \tag{152}$$

The bracket equals the nonzero constant $c_{\alpha,s;\sigma(j_\alpha)}$ at $x = 0$. All strict transforms of such denominators are therefore units; their pullback divisors consist only of the coordinate hyperplanes of $x_{j_\alpha}, \ldots, x_{c_V}$. A hyperplane that recurs is the same irreducible divisor and does not increase the number of normals, so near the boundary there are at most c_V mutually independent coordinate normals. This proves: as long as the denominators of the dlog and top-to-sub formulas keep the affine-linear structure of Eq. (150), the blow-up corresponding to a strict ordering of all τ_i eliminates the degenerate singularities caused by all $K_i \to \infty$ occurring simultaneously. The proof is independent of whether the original propagator graph is a tree or a time-only loop graph.

The program must still provide a certificate and cannot rely only on the above formal check of the top sector. A simple and sufficient algorithm is: collect all irreducible divisors of the complete block-triangular connection, the normalization gauge and the coordinate Jacobian; substitute Eq. (140); extract all monomial powers of the x_j; and require the remaining strict transforms to be nonzero at $x = 0$. If some strict transform still vanishes, check, for the deduplicated local defining functions $f_1, \ldots, f_m$,

$$\nabla f_i \neq 0, \qquad \operatorname{rank}\left(\frac{\partial(f_1,\ldots,f_m)}{\partial(x_1,\ldots,x_{c_V})}\right) = m \leq c_V. \tag{153}$$

If Eq. (153) is not satisfied, the singularity is still a degenerate multi-variable one. For the letters of Eq. (150), Eq. (152) already proves that all strict transforms become units; the check in the program is a certificate for the formula implementation and for the extra gauge/normalization, not a second guess of this conclusion.

The public interface `MSBoundaryChartCertificate` implements this certificate. It collects all sector letters of `MSDLogDE`, the normalization divisors and the shifted-contact recurrence singular layers, and returns for each divisor the dominant rank, the coordinate monomial, the strict transform and its value at $x = 0$; it also returns the complete Jacobian from the user energies to K_v and then to x_j, the per-sector theta-fixing table, and the $P_s S_s = I$ quotient check. `RankOrder->All` enumerates all root strict charts. `MSBoundaryData` forces consumption of the certificate of the chosen chart before constructing the infinity Frobenius data and the finite matching points; any non-affine new denominator, vanishing strict transform, zero Jacobian or uncertified quotient returns `BoundaryChartNotCertified` and does not continue the numerical transport.

The theorem above also fixes the scope of automation. Singularities that contain no K_v at all are already listed explicitly in the DE letters; they are not obstacles for this boundary producer. An ordinary polyline can use `"Avoid"` to refuse a crossing. For an explicitly supplied

finite singular user point, `"Automatic"` uses the backend power–log basis to classify a finite limit, a pole, or a logarithmic divergence and uses the same local basis for user points on both sides inside its convergence disk. More general blow-ups, non-collinear turning at a singularity, and new denominators introduced by an extra gauge transformation do not automatically inherit the proof above.

The boundary condition of each sector must treat that sector itself as the top sector: use the post-contact component base power, the normalization and Eq. (148). Compute the bottom sectors first, then move up along the sector DAG. After theta fixing, the homogeneous leading term factorizes into a product of component vertex-family boundary coefficients; but the supersector may also receive a leading contribution from the lower-sector solution. In the five-dimensional example of Sec. 4.2 of the reference, the remaining integral already has nonzero leading coefficients on the second and third components of the original sector, which is exactly why writing only the homogeneous product misses terms. A general producer should solve the block-triangular Frobenius leading system, or equivalently keep the small-time expansions of the theta/delta, to obtain the complete same-ordered boundary vector recursively.

12.5 Controlled transport of an auxiliary external-leg energy to zero

From Eq. (51),

$$\det \mathrm{M}_{0;C,s} \propto \prod_{\boldsymbol{a},\boldsymbol{r}} \kappa_{C,s;\boldsymbol{a},\boldsymbol{r}}. \tag{154}$$

Here $\mathrm{M}_{0;C,s}$ is the momentum-layer matrix of component C in sector s (the subscript C marks the component and s the sector); after common diagonalization its diagonal entries are $\mathrm{i}\kappa_{C,s;\boldsymbol{a},\boldsymbol{r}}$, so the determinant is proportional to the product of these letters. So the raising recurrence (71) fails on any $\kappa = 0$. The smallest counterexample is a single (anti-)time-ordered propagator $e = (u, v)$: the top-sector letters are

$$p_{0,u} \pm q_e, \qquad p_{0,v} \pm q_e, \tag{155}$$

which, for generic $q_e \neq 0$, do not vanish even when the two auxiliary energies are set to zero; but the sector with the pinched propagator has no such two-dimensional slot and necessarily contains

$$\kappa_{\{u,v\}} = p_{0,u} + p_{0,v}. \tag{156}$$

Therefore, when these two p_0 consist only of the auxiliary energies to be removed, the complete top/subsector system is singular where the auxiliary energies vanish simultaneously. More generally, any reachable component that, at the target point, neither provides an active slot with nonzero signed energy nor has a nonzero total exponential energy necessarily has a zero letter.

The general conclusion for $\mathrm{M}_1 = 0$ has already been given by Eqs. (75)–(77): a general mixed state must consult the asymptotics at the end of the path separately; the pure-massless exception was explained after Eq. (76), where $\mathrm{M}_1 = 0$ corresponds exactly to the divergence of the unregularized $\Gamma(\alpha_v)$ and an analytic regulator must be introduced first. The boundary and recurrence producers should consume the regularized generic parameter system, and must not misreport an unregularized zero eigenvalue as a new master or a physical failure of the recurrences.

If the target point itself lies on $\mathrm{M}_0 = 0$, one must first reduce at generic vertex energies and then take a controlled limit, or the user must handle that singularity separately using the output DE; a zero point must not be handed to the ordinary-point numerical transport.

The recommended flow is therefore: first choose the preferred chart according to Eq. (146), obtain the same-ordered vector from the boundary of Eq. (140), take a small but finite ordinary point inside the chart, and then let the user provide a multivariable polyline in the required homotopy class to the preferred ordinary point and the final target. `"Avoid"` reports a direct crossing of a letter hypersurface, `"Automatic"` classifies an explicitly supplied singular user point and resumes from an outgoing ordinary match on the same straight continuation branch, and `"SingularityJump"` explicitly selects the local crossing branch and requires confirmation of its

multivaluedness. Auxiliary vertex energies may be taken to zero one at a time; a finite singular point returns either a finite value or `Infinity`. A singularity at a complex-affine turn still fails closed because the input does not uniquely specify a continuation branch.

12.6 The [2] infinity boundary and FlintNDE transport

The production entry point does not compute the defining integral directly at a finite Euclidean ordinary point. `MSBoundaryData` uses the $K_v \to \infty$ data of [2] as the only boundary source. If a vertex has no physical external leg, the program internally introduces a private auxiliary external-leg energy $k_{0,v}^{\mathrm{aux}}$ in the same analytic dlog system, first constructs the boundary data from the corresponding damped limit $K_v^{\mathrm{aux}} \to \infty$, and then uses FlintNDE to transport along that auxiliary coordinate to the physical target $k_{0,v}^{\mathrm{aux}} = 0$. The user does not enter this private coordinate in `pointSequence` or `ParameterRules`. Every closed tree/time-only context, including the single-vertex massive-external family, uniformly consumes the sector DAG, components, slot registry, normalization, state order and strict rank and follows the "boundary leading term plus FlintNDE Frobenius recurrence" route. The explicit multivariable series of Eqs. (3.44)–(3.46) of [2] is retained only as a test oracle and is not a production dispatch.

The boundary request also ships, letter by letter, the coprime numerator and denominator coefficients of each letter's pullback in t together with its tensor contributions. Pullbacks independent of t are skipped, and the exact rational data never pass through a symbolic extension field. The bundled backend isolates the roots of numerator and denominator factors of degree at most three as certified decimal balls, assembles the residues $\pm$ multiplicity $\times R_\ell$ pole by pole, and switches the Frobenius transport to that pole recurrence after checking agreement with the dense rational system at path probes; any failure of isolation, assembly or probing falls back to the general sampling route with unchanged results.

Concretely, for a component C of a sector s define (every ν in this subsection is the parameter of this handbook after Eq. (4))

$$\begin{aligned} \alpha_{sC}(\boldsymbol{a}) &= \nu_{0,C} + 1 - \sum_{r \in H(s,C)} a_r(1 - 2\nu_r), \\ \ell_{sC} &= V - \min_{u \in C} \operatorname{rank}(u) + 1, \qquad K_{sC}(t) = c_{sC} t^{-\ell_{sC}} + O(t^{-\ell_{sC}+1}). \end{aligned} \tag{157}$$

Here α_{sC} is the boundary exponent of component C, with the subscript s marking the sector and C the component; $K_{sC}(t)$ is the leading factor of that component at the common small boundary time $t \to 0$ (not the damping energy K_v), ℓ_{sC} is its leading power, c_{sC} is the coefficient of that leading term, and V is the number of vertices. Here $H(s, C)$ contains only the still-active Hankel slots of massive propagators on that component; massless shared slots do not enter α_{sC}, but instead give a ± 1 state factor according to the fixed theta chamber. The Frobenius exponent and the physical leading weight of the corresponding master are

$$\begin{aligned} \lambda_{s,\boldsymbol{a}} &= \sum_C \ell_{sC} \alpha_{sC}(\boldsymbol{a}), \\ W_{s,\boldsymbol{a}} &= \mathcal{N}_s \Xi_{s,\boldsymbol{a}} \prod_C \left[(-\mathrm{i})^{\nu_{0,C}+1} \Gamma(\alpha_{sC}) E_{sC,\boldsymbol{a}}\, c_{sC}^{-\alpha_{sC}} \right]. \end{aligned} \tag{158}$$

Here $\lambda_{s,\boldsymbol{a}}$ is the Frobenius exponent of master $(s, \boldsymbol{a})$ and $W_{s,\boldsymbol{a}}$ is its leading weight. $\mathcal{N}_s$ is the sector normalization, $\Xi_{s,\boldsymbol{a}}$ collects the shared-massless chamber sign (the subscript s marks the sector and $\boldsymbol{a}$ the state bits), and $E_{sC,\boldsymbol{a}}$ is the product of all Hankel-block coefficients of the component (the subscript sC marks component C of sector s), where a massive (anti-)time-ordered propagator whose two endpoints already lie in that same component enters through the mean of its two arrangements as in Eq. (165). Each pinched massive (anti-)time-ordered propagator e contributes the pinch factor of Eq. (2.17) of [2],

$$-\frac{4\mathrm{i}}{\pi} \eta_e k_e^{2\nu_e - 1}, \qquad \eta_e = \begin{cases} +1, & ++ \\ -1, & -- \end{cases}, \tag{159}$$

to $\mathcal{N}_s$, where η_e is the SK sign of propagator e (the subscript e marks the propagator). The factor η_e defines only the normalization of the physical subsector master; it is not multiplied again into the normalized-master dlog contact block or recurrence event. The remaining constant coefficient of a massive contact is already complete in the defining-integral coefficient of each component, and no extra phase may be multiplied according to contact depth. All quantities are taken from the current sector metadata; one must not guess the subsector powers or two-state factors from the root sector.

Pull back the complete dlog potential along $K_{\sigma(j)} = B^{V-j+1}t^{-(V-j+1)}$ and denote the residue by R_0, where $t \to 0$ is the common small boundary time and $B > 1$ is a fixed reference scale. The construction uses the constant-residue form $\Omega = \sum_\ell R_\ell \log \ell$ directly. Since R_ℓ is independent of the external energies, the curve enters only the scalar letters and

$$R_0 = \sum_\ell \rho_\ell R_\ell, \qquad \rho_\ell = \lim_{t\to 0} t\frac{\mathrm{d}}{\mathrm{d}t}\log \ell(t). \tag{160}$$

Curve substitution, differentiation and the limit are therefore performed separately on every scalar letter; only the remaining numerical rules are substituted into the residue matrices, with no such operation on the full symbolic Ω matrix.

For each $(s, \boldsymbol{a})$, the program fixes the component of the corresponding root master in that sector to 1 and the components of the non-ancestor sectors to 0, and solves

$$(\lambda_{s,\boldsymbol{a}}\mathbb{I} - R_0)v_{s,\boldsymbol{a}} = 0. \tag{161}$$

where $v_{s,\boldsymbol{a}}$ is the leading vector of that master and R_0 is the residue obtained by the pullback above. Then $W_{s,\boldsymbol{a}}v_{s,\boldsymbol{a}}$ automatically contains the leading components injected into the supersector by the subsector solutions. Repeated roots or integer-difference resonances do not create graph-specific parameters; the complete exact residue together with these $\{\lambda, 0, v\}$ is handed to the power-log recurrences of FlintNDE. Finite defining integrals are independent-validation oracles only and never production inputs.

The same curve makes every sector diagonal block explicitly diagonal. Equations (87), (157) and $\mathrm{d}\log\kappa_{sC} \sim -\ell_{sC}\mathrm{d}t/t$ give

$$R_{0;ss} = \sum_C \ell_{sC}\mathrm{M}_{1;C,s}(\nu_{0,C} + 1) = \mathrm{diag}_{\boldsymbol{a}}(\lambda_{s,\boldsymbol{a}}); \tag{162}$$

neither $\Omega_{\mathrm{ex},s}$ nor the normalization gauge has a pole along this curve, so neither enters $R_{0;ss}$. After fixing v_r in the root sector r, the remaining blocks follow by back substitution in reverse contact-DAG order:

$$v_{s,\boldsymbol{b}} = \frac{\left(\sum_{t\succ s} R_{0;st}v_t\right)_{\boldsymbol{b}}}{\lambda_{r,\boldsymbol{a}} - \lambda_{s,\boldsymbol{b}}}, \qquad s \neq r. \tag{163}$$

The side length of $R_{0;ss}$ is the master count of sector s, while $R_{0;st}$ has the master counts of sectors s and t as its two dimensions. Every block has its own binary-bit space and quotient; the global system is not one 2^n-dimensional tensor space. Equation (163) is used only when every denominator is nonzero and the residual in the root rows also vanishes. A zero denominator or a nonzero residual falls back to the full constrained solve in Eq. (161). The directly substituted branches therefore need no generalized power series, while any logarithmic or Jordan structure remains a decision of the subsequent FlintNDE recurrence.

The necessity of logarithms must not be guessed from repeated roots or from a fixed number of subleading orders. The minimal exact check has two layers. First, compare the algebraic multiplicity m_α with the geometric multiplicity $g_\alpha = \dim\ker(\alpha\mathbb{I} - R_0)$ for each α; only $g_\alpha < m_\alpha$ means the root carries a Jordan chain and hence a leading logarithm. An independent-leading-vector count below g_α merely signals missing leading data, not a Jordan defect, and a vanishing physical branch coefficient does not remove a solution. Second, only positive integer root differences require compatibility checks at the corresponding resonant orders. If all geometric

multiplicities agree with the algebraic ones and no such difference exists, the pure-power solutions are already complete and no subleading coefficient is needed. Parameter degeneracy occurs on the exceptional set where these rank or compatibility conditions fail; away from that set the original analytic pure-power expressions are unchanged, and the same criterion decides whether extra logarithms are required at a special point.

Both layers are carried by the power-log certificate attached to the boundary data and shipped with the evaluation request: the certificate records the exact value, algebraic multiplicity, geometric multiplicity, Jordan defect and positive-integer differences of every α. The bundled backend builds a verification-only manifest from it: for each claimed root, ascending kernel chains confirm that the generalized eigenspace dimension equals the claimed multiplicity, and the multiplicities sum to the master dimension; together these uniquely determine $\det(\lambda\mathbb{I} - R_0) = \prod_\alpha (\lambda - \alpha)^{m_\alpha}$, so the backend no longer solves a characteristic polynomial or searches for roots. When logarithms or mixed-defective Jordan structure appear, the power-log recurrence proceeds inside the known spectrum: spectral projectors separate the root subspaces, nonresonant orders are solved top-down in the log degree with a cached inverse $(\gamma\mathbb{I} - R_0)^{-1}$, and each resonant order assembles all of its log degrees into one block system, raising the log budget by one on incompatibility, so the first appearance order of a logarithm emerges naturally; the program never falls back to a spectrum-recomputing entry point because of repeated roots, integer differences or logarithms.

When an input coefficient or a certificate root is not an element of $\mathbb{Q}(\mathrm{i})$ — for example a radical from a parameter rule — the serialization gate converts it automatically to a working-digit decimal instead of rejecting the request, while every $\mathbb{Q}(\mathrm{i})$ value is still serialized byte-identically as an exact string; non-numeric symbols keep failing closed. The backend parses such decimals as exact rationals, so the system becomes an exactly-rational perturbation of relative size about $10^{-\text{digits}}$, and the only obstruction is the spectrum gate itself. In that case the gate raises a typed error and the adapter reruns the whole launch chain — path planning, the letter fast system and the transport — through the numerical Frobenius route: the certificate spectrum, decimalized to the requested working digits, is cross-checked root by root (value, multiplicity and total count) against the backend's independent ball-eigenvalue clustering, the matching of the $\{a, b, C\}$ boundary records is repeated with cutoff-controlled linear solves, and the report records which route was used. Every retained digit count in this chain derives from the requested working precision — the certificate roots are decimalized at the working digits and the numerical gate declares half of them as its reliable precision, which keeps the zero cutoff safely above the decimalization perturbation — so raising the user's precision request tightens the whole chain proportionally. The numerical route triggers only for non-$\mathbb{Q}(\mathrm{i})$ spectra; $\mathbb{Q}(\mathrm{i})$ requests run the exact path unchanged.

Writing only one power of τ per component above is not an omission but the counterpart of the two-power structure of h and of the two states. Denote by $\mathsf{c}_a^{(\mathrm{b})}$ the entry of `msVertexEndpointCoefficient` at Hankel branch $\mathrm{b} \in \{1, 2\}$ and state bit $a \in \{0, 1\}$, and let $s_\nu = \pm 1$ be the prefactor convention (`formulaNu` $= s_\nu \nu$). The two Frobenius towers at small z are

$$h^{(\mathrm{b})}(\nu, 0; z) = \mathsf{c}_0^{(\mathrm{b})}\big(1 + O(z^2)\big) + \frac{\mathsf{c}_1^{(\mathrm{b})}}{2s_\nu\nu}\, z^{2s_\nu\nu}\big(1 + O(z^2)\big), \qquad h^{(\mathrm{b})}(\nu, 1; z) = \mathsf{c}_1^{(\mathrm{b})}\, z^{2s_\nu\nu - 1}\big(1 + O(z^2)\big). \tag{164}$$

The constant term disappears under differentiation, so the state with `bit`$= 0$ consumes the first power and the state with `bit`$= 1$ consumes the second one: the two power starts are not two alternative branches between which one chooses, but one power per state of the same master. Subtracting $a_r(1 - 2\nu_r)$ for every active slot in Eq. (157) is precisely the bookkeeping of this fact — the ν_r there is `formulaNu`$_r$ and already carries the sign s_ν, so switching a bit from 0 to 1 moves α_{sC} by $2\nu_r - 1$, the exponent difference of the two powers in Eq. (164). The eight entries themselves are fixed by the small-argument expansions of $J_{\pm\nu}$ and are identities for generic ν; they are not dispatched according to the real or imaginary character of ν. Eqs. (3.52) and (3.53) of [2], which read as one set for real ν and another for imaginary ν, are the same identity written

once in ν and once in $\bar{\nu}$. When the exponent difference of the two powers is an even integer the two towers coincide, the corresponding masters then carry the same λ, and that is exactly the resonance case handed to the power-log recurrences of FlintNDE in the previous paragraph; the boundary data do not gain a row.

The number of independent Frobenius solutions of the differential equation is therefore the number of masters, and each solution is fixed by the leading record of the master it anchors; no master needs a second record. In the single-vertex two-master example of [2], Eq. (3.10) gives $C(2,0) = 0$ for $\lambda = \nu_0 + 1$ and $C(1,0) = 0$ for $\lambda = \nu_0 - 2\nu_1$: the two exponents are the two powers of Eq. (164), and in each solution the component of the non-anchoring master vanishes at leading order. Eq. (3.40) of the four-master case shows in addition that two solutions may share one exponent while anchoring different masters. The program produces one record per master through `rootIndex`, and Eq. (161) fixes the root component to 1, the non-ancestor components to 0 and solves the remaining components linearly, which is term-by-term the structure of those two indicial systems. The criterion stated by that paper right after Eq. (3.16) is the reason for keeping only the leading term: the $O(\tau^{-2\nu})$ term of $h^{(2)}(0, -k\tau)$ contributes only to the next-to-leading term of solution two, so its coefficient is not needed at the boundary. MadStree likewise leaves the higher powers of τ, the logarithms and the Jordan structure to the power-log recurrences, and its boundary data carry no subleading row; for $\nu = \mathrm{i}\mu$, where the two powers have equal modulus, the assignment is decided by the exponent λ and not by the modulus, so the conclusion is unchanged.

This route closes only if every retained exponent has a nonzero anchor, and the producer guards that with two gates. If $W_{s,\boldsymbol{a}} = 0$ for a master, the record anchors no solution and is dropped, and the value of that component is supplied by the leading vector of another root instead. The solution and the residual of Eq. (161) are checked component by component; if they fail, the producer returns `SectorLeadingSystemFailed` or `SectorLeadingSystemResidual` rather than treating a system without anchors as a determined boundary condition.

The assignment of the two powers considered so far concerns only the interior of one Hankel branch. One independent choice is still open inside $E_{sC,\boldsymbol{a}}$: which branch b each endpoint of an internal propagator sits on. When the two endpoints of a massive (anti-)time-ordered propagator lie in two different components, its theta is frozen to a definite 0 or 1 by the strict energy order and the assignment of the two branches agrees with the chamber choice of Eq. (79), so there is nothing left to choose. Once the two vertices have been pinched into a single component, that propagator carries no theta of its own: its integrand is just the product of the two h blocks at its endpoints, and the ordered pair $(\mathrm{b}_r, \mathrm{b}_{r'}) = (1, 2)$ supplied by the strict order and the other arrangement $(2, 1)$ are two arrangements of one and the same equal-time fibre, with the same α_{sC} and the same momentum powers. A defining integral must be constant along that fibre, so the program takes the arithmetic mean of the two arrangements for such a propagator: writing r, r' for its two endpoint slots, the branch factor it contributes to $E_{sC,\boldsymbol{a}}$ becomes

$$\mathsf{c}^{(\mathrm{b}_r)}_{a_r}\,\mathsf{c}^{(\mathrm{b}_{r'})}_{a_{r'}} \;\longrightarrow\; \tfrac{1}{2}\Big(\mathsf{c}^{(1)}_{a_r}\mathsf{c}^{(2)}_{a_{r'}} + \mathsf{c}^{(2)}_{a_r}\mathsf{c}^{(1)}_{a_{r'}}\Big), \qquad e \text{ coincident inside } C, \tag{165}$$

where all three c entries take the Hankel order of that propagator, so the two endpoints of one propagator share a single ν; every propagator that is still strictly ordered keeps its original factor. This is the same symmetry that makes the spectators of Eq. (82) contribute $\mathsf{J}_e = (A_e + B_e)/2$: the antisymmetric part D_e survives only on the theta that is being differentiated, and an equal-time propagator keeps only its symmetric part. The mean acts on the branch index alone and the two state bits are untouched, so α_{sC}, $\lambda_{s,\boldsymbol{a}}$ and $\mathcal{N}_s$ are all unchanged, and the only thing rewritten in Eq. (158) is a constant factor of $E_{sC,\boldsymbol{a}}$; the momentum powers $(-\mathrm{i}q_e)^{-a(1-2\nu_e)}$ do not depend on the branch and stay inside $E_{sC,\boldsymbol{a}}$ slot by slot. When $a_r = a_{r'}$ the two arrangements give the same product and the mean coincides with the previous factor word for word, so the mixed-bit arrangements are the only ones changed by Eq. (165).

With arbitrarily many propagators joining the two vertices this rule still applies one propagator at a time: n coincident propagators contribute n such mean factors multiplied together, which is exactly the per-propagator tensor product that the odd-subset algebra requires rather than one

mean taken over the whole bundle — J appears once per propagator in $\prod_{e\in S} D_e \prod_{e\notin S} \mathsf{J}_e$, and the boundary mean appears once per propagator as well. Two cases are deliberately not averaged. The two branches of a massive Wightman propagator (contour "+−" and "−+") are already fixed at initialization by `hankelBranches`, so such a propagator has no theta to freeze; and an equal-time massless shared slot carries no branch choice at all, its odd shared state being zero by the equal-time rule recalled after Eq. (82).

Eq. (4.2) of [2] gives the remaining term of the two-vertex case with equal time powers ν_0. Written in the $+\nu$ convention of this handbook via Eq. (4), it becomes the following, in which the superscript paper only records that the basis of states at the two vertices used by that paper is still in use, with conjugate Hankel orders at each of them, and does not denote another prefactor convention:

$$I_R^{\text{paper}} = -\frac{4\mathrm{i}}{\pi} e^{-\pi \operatorname{Im}\nu_1} k_s^{2\nu_1-1} \int_{-\infty}^{0} \mathrm{d}\tau\, (-\tau)^{2\nu_1-2\nu_0} e^{\mathrm{i}(k_{12}+k_{34})\tau}. \tag{166}$$

where k_s is the momentum scale of that example and k_{12} and k_{34} follow that paper's notation as momentum combinations of the two external-leg groups. This expression cannot be copied directly into the MadStree sector normalization without a basis conversion. For a heavy physical order, whose Hankel orders are $\pm\mathrm{i}\mu$ with μ real, the conjugate-order states at each vertex used by [2] and the common-order states at that vertex used by MadStree obey

$$e^{-\pi\mu} H^{(2)}_{-\mathrm{i}\mu}(z) = H^{(2)}_{\mathrm{i}\mu}(z). \tag{167}$$

The explicit Boltzmann factor in Eq. (166) is therefore absorbed by the conversion between the two bases at the vertices. This example is G_{++}, so Eq. (159) has $\eta_e = +1$ and the subsector normalization in the common-Hankel-order MadStree basis is

$$\mathcal{N}_R^{\text{MS}} = -\frac{4\mathrm{i}}{\pi} k_s^{2\nu_1-1}, \tag{168}$$

where the superscript MS marks the MadStree convention and the subscript R marks the remaining-term subgraph; it must not be multiplied by $e^{-\pi \operatorname{Im}\nu_1}$ again. The same fifth branch has the leading vector and the physical coefficients of the two bases at the vertices

$$\begin{aligned} v_5 &= \left(0, \frac{1}{\nu_0-2\nu_1}, \frac{1}{1-\nu_0}, 0, 1\right)^T, \\ C_5^{\text{paper}} &= -\frac{4\mathrm{i}}{\pi} e^{-\pi \operatorname{Im}\nu_1} k_s^{2\nu_1-1} e^{-\mathrm{i}\pi(2\nu_1-2\nu_0+1)/2} \Gamma(2\nu_1-2\nu_0+1), \\ C_5^{\text{MS}} &= -\frac{4\mathrm{i}}{\pi} k_s^{2\nu_1-1} e^{-\mathrm{i}\pi(2\nu_1-2\nu_0+1)/2} \Gamma(2\nu_1-2\nu_0+1). \end{aligned} \tag{169}$$

where the subscript 5 of v_5 and C_5 marks the fifth branch listed by that paper, and the superscripts paper/MS mark that paper's printed vertex basis and the common-Hankel-order MadStree basis, respectively. This uses the $i\epsilon$ branch $\int_{-\infty}^{0} \mathrm{d}\tau\, (-\tau)^p e^{\mathrm{i}K\tau} = (-\mathrm{i})^{p+1}\Gamma(p+1)K^{-p-1}$. The phase of Eq. (4.11) as printed in [2] reads $e^{\mathrm{i}\pi(\nu_0-\nu_1)}$ in the convention of this handbook and is larger by one factor of i than this direct evaluation of the defining Eq. (166). MadStree follows its public definition $J_s = \mathcal{N}_s \mathrm{I}_s$, uses the direct integral, and explicitly maps to the same basis at the vertices before comparison with [2]. The base formula stored in the program is exactly this $+\nu$ form (the order slot reads `formulaNu`$= s_\nu \nu$, by default $+\nu$). The printed formulas of [2] differ from the formulas above only by the order substitution $\nu_{\text{ref}} = -\nu$, which is already incorporated in them. The five-dimensional result above is now merely a check example of Eqs. (157)–(161), not a production dispatch; general unequal time powers read the $\nu_{0,C}$ of each component directly. The massive pinch normalization for real positive physical momenta is fixed by Eq. (168); the formal symbol $(-k_s)^{2\nu_1-1}$ must not be handed to the Mathematica principal branch.

`MSEvaluatePath` partitions the input order into maximal consecutive complex-affine one-variable segments $x(s) = x_0 + sv$ and pulls each segment back only once. It sends every exact

complex parameter point, the boundary, and the selected options in one UTF-8 request. MadStree neither selects nodes nor builds detours; FlintNDE initializes the boundary, optionally plans, and transports in one process. With planning enabled, points covered by one expansion node form an evaluation bucket: large buckets use subproduct/remainder-tree fast multipoint evaluation and small buckets use the iterative algorithm. With planning disabled, every user point is a transport node. The second argument is always named `pointSequence`. Its first row is the ordered header of running-coordinate symbols and every later row is an equal-width value row. A single point has one value row; multiple points only append rows. Fixed, non-running parameters are supplied once through `ParameterRules`. The header and fixed parameters must be disjoint and together cover every symbol required by the analytic DE and boundary. Missing or still nonnumeric values return a Failure listing the symbols before a runtime directory is created or Python/FlintNDE starts. The function, options, backend schema, `pointResults`, and return structure do not depend on the number of points. `TransportOrder` sets the production primary chain; nodes, dense points, the ends of segments, and regulator-fit samples all come from this lower-order chain. The higher `ReferenceTransportOrder` chain is used only for the relative-difference accuracy check. Regular-singular initialization and successive affine segments propagate both states independently, so the end point of a reference chain neither replaces user output nor restarts the next primary chain. Distinct segments never share local coefficients. A single project-relative configuration variable controls the backend location. `MSRuntimeDirectory` denotes the temporary runtime root itself; `Automatic` creates one `results_temp/` next to the calling script, with short `nde/` and `cache/` children, never in the package source tree. On Windows, complete cache, input, and log paths are checked before directory creation or Python launch. Paths longer than 259 characters return the separate `RuntimePathTooLong` failure. Input writes, missing `python-flint`, launch failures, missing output, and invalid output retain distinct tags. CSV/JSON export returns `"written"` only when every requested format has been written successfully.

The recurrence on ordinary affine segments executes in tensor-product blocks. The main line always transports the complete vector, retaining all sectors and sources; when an operator term carries an identity spectator factor (of the form $B \otimes I_n$), the backend rearranges the full solution at that order into a matrix Z of active rows times spectator columns and applies BZ directly, reducing the matrix-multiplication cost from the dense upper bound $O(m^2n^2r)$ to $O(m^2nr)$, where m is the active dimension, n the spectator dimension, and r the number of solution columns. Identity factors are recognized at construction time from the exact $\mathbb{Q}(\mathrm{i})$ nonzero pattern — a paired bit must be preserved and its coefficient must be equal before and after the flip — after which only the active matrix and a gather/scatter layout are stored. Different terms may have different or overlapping active supports and are applied separately and summed; rectangular source blocks use the same row–column pairing layout and are neither zeroed out nor required to commute with other terms. Each letter contribution keeps its own support; contributions with the same pole and output layout are merged in exact arithmetic, and different input blocks with the same output layout are combined into one block multiplication. Pole states accumulate order by order as $w_p[n] = (w_p[n-1] - B_p Z[n])/(p - \mathrm{center})$; an ordinary segment neither builds a global dense residue nor applies operators slot by slot over 2×2 factors. Boundary rational systems certified as a polynomial plus $\mathbb{Q}(\mathrm{i})$ simple poles use the same block partial-fraction recurrence, while all other cases keep the general analytic route; the Frobenius/power-log initialization and the known-roots handling are unchanged. Spectator components still participate in the recurrence — they simply do not undergo the mixing absent from that operator term — and blocks with non-binary quotients or without a separable identity factor keep a general rectangular active matrix. The block organization changes only how the matrix products are scheduled, never a numerical result.

The bundled backend has two execution lanes. The default Python lane performs boundary initialization and transport in one process through the `python-flint` certified dual chain. When a request satisfies the engine contract — `FlintNDEPathPlanning -> False`, `SingularityMode -> "Avoid"`, a finite boundary, and no segment start or user point on a letter pole — the adapter hands the whole request to the bundled C++ engine (`Vendor/FlintNDE-dS/engine/`) with an un-

changed result structure. A missing engine or any unsatisfied condition falls back to the Python lane automatically; setting `MADSTREE_FLINTNDE_ENGINE` to `python` disables the engine lane, an executable path overrides the probe location, and `FLINTNDE_ENGINE_LIBPATH` names the libflint runtime directory. Regular-singular boundaries (including the three-vertex zd production boundary) and planning-enabled requests stay on the Python lane by contract, and the `engine` field of each segment's `planReport` reports the lane honestly.

The analytic `MSDLogDE[context]` is itself a durable user result. A symbolic-only workflow may call `MSWriteFormulaArtifacts` explicitly. `MSEvaluatePath` and `MSReconstructEpSeries` automatically write or reuse the complete formal artifacts before any Python/FlintNDE task starts. A write failure prevents numerical NDE, and the successful result reports every path under `"formulaArtifacts"`. The default files are `masters.wl`, `recurrence_metadata.wl`, `dlog_de.wl`, and `manifest.wl` under `results/madstree_formula/run-<UUID>/` by default. The dlog file always contains the complete analytic DE. Numerical work derives only a function-local parameterized copy and never overwrites the analytic object or durable file. Repeated evaluations in one session reuse the complete files for the same context and calling root; deleted files are regenerated, while any partial write fails before backend launch. Ordinary-point export defaults to `results/madstree_evaluation/run-<UUID>/evaluation_data.csv/json`. `results/` contains durable user outputs, `results_temp/` contains requests, logs, and cache, and `test/results_test/` contains development-test artifacts.

Every multipoint transport through intermediate nodes is a jump; only a crossing of a singularity through a local-basis connection is a singularity jump. The default `SingularityMode -> "Automatic"` follows explicitly supplied singular user points and warns about the local branch. An exact singularity is removed from the ordinary-node chain; one local basis owns user points on both sides inside the disk whose radius is the distance to the nearest other singularity. A removable singularity returns a numerical value, while a true pole or a zeroth-order logarithmic divergence returns the text `Infinity`; transport then resumes from the outgoing ordinary match. A terminal singular target receives a hidden in-disk ordinary match point along the incoming direction when required. The lower-order primary chain supplies the result, and the higher-order reference chain only checks classifications and finite values. The evidence is listed separately in `singularityClassifications`. `"Avoid"` explicitly refuses a crossing, whereas `"SingularityJump"` explicitly selects the local crossing branch and warns the user to confirm the multivalued branch of the equivalent detour class. A non-collinear singular turn, consecutive singular points, and an intermediate singular point with planning disabled remain fail closed. `MessageLanguage -> "EN"` gives English notices by default, while `"CN"` selects Chinese; the values are case-sensitive. The backend discovers finite singularities from each pulled-back one-variable system, so the user does not supply a list.

If every dlog letter is constant on a segment, every pulled-back derivative vanishes, so the segment is a zero connection with no finite poles and is transported normally. Both `MSBoundaryData` and `MSEvaluatePath` default to 200 decimal working digits; automatic regulator planning takes the larger of 200 and its adaptive estimate, while an explicit precision overrides the default. FlintNDE uses $\lceil p_{\mathrm{dec}} \log_2 10 \rceil + 32$ working bits: the 200-digit default gives 697 bits, while 70 and 100 digits use 265 and 365 bits, and higher requests continue to grow by the same formula; the numerical gates of the local singular basis tighten with the current precision as well, and the retained serialization widths and tolerances derive from the same working digits instead of fixed constants. Segment projection, match ratios, rotation factors, and winding/monodromy retain the current Arb precision.

12.7 Symbolic Laurent-support certificate and regulator error control

Let the shared regulator be ϵ. In `MSReconstructEpSeries[..., MaximumEpPower -> m]` the user specifies only the highest returned power m; the lowest power $n_{\min}$ is not a numerical fit parameter. Before starting any NDE task at nonzero ϵ, MadStree constructs a support certificate

from the physical boundary and dlog DE of the same production context. It requires:

1. regulator-independent path coordinates and no negative Laurent power at $\epsilon = 0$ in any dlog matrix, pulled-back connection, or residue;

2. finite-Laurent defining-integral boundary branches; branches with the same limiting Frobenius exponent and log degree are first combined with their physical weights and leading vectors, and only then are component valuations extracted;

3. at least one provably nonzero coefficient at the lowest power of the combined boundary. A noninteger power, non-Laurent structure, undecidable coefficient, or support entirely above the checked user range fails closed.

Before these symbolic valuations, the package also checks that the numerical input is complete and that a generic boundary chart can be constructed. This coverage preflight uses the fixed nonzero exact probe $\epsilon = 1/1009$, so a legitimate normalization pole is not rejected by a generic numeric-completeness check. The probe is neither a production nor a validation sample, never enters the fit, and does not determine $n_{\min}$; the dlog DE, complete physical boundary, and resonance records are still Laurent-valued strictly at $\epsilon = 0$. For $n_{\min} < 0$ the certificate describes a finite meromorphic Laurent boundary, not an analytic boundary at $\epsilon = 0$; analyticity is recorded only when $n_{\min} \geq 0$. Along the current finite path, an ϵ-analytic invertible fundamental matrix preserves the valuation of nonzero formal boundary data, so the certified $n_{\min}$ is passed directly to numerical sample planning. A rank drop of an individual Frobenius recurrence operator at $\epsilon = 0$ is retained as a resonance diagnostic only; it is not by itself interpreted as a $1/\epsilon$ pole across the already combined physical boundary.

Example 06 uses the massless three-vertex family with $a_1 = -1 + \epsilon$ and $a_2 = a_3 = 1 + \epsilon$, whose $\epsilon = 0$ time integrals genuinely diverge. The vertex-1 unit $u_1 = a_1 + 1$ equals ϵ and is isolated as a standalone factor in part of the factorized boundary weights, and every such branch carries a single $\Gamma(\epsilon)$ simple pole. The actual nine-dimensional DE still has no negative ϵ power. The program therefore certifies $n_{\min} = -1$ before any NDE solve, from the combined physical boundary: the pole comes from the boundary data itself and not from any individual resonance.

After support certification, the square interpolation initially includes at least two powers above the user maximum. Independent validation points never enter the fit. If their relative residual exceeds the fixed `EpGoalDigits` target, the next round adds two fitted powers by default, solves only the new production points, and reuses every previous production and validation value. If the target is still missed after at most three rounds, the current best coefficients are returned without precision certification: the tolerance is not relaxed, least squares is not substituted, and caches are not discarded. Independent regulator tasks use an outer process pool with default `ParallelTaskCount -> 12`; the effective count is the smaller of this limit and the number of tasks.

The default call still contains no regulator values. To restrict every solve to an approved range, `EpSamplePoints -> {...}` supplies an ordered production candidate pool and `EpValidationPoints -> {...}` supplies an independent grid disjoint from that whole pool. The pool may be redundant. With `EpInitialInternalMaximumPower -> q`, the first round consumes exactly the $q - n_{\min} + 1$ candidates required for powers $n_{\min}, \ldots, q$; later candidates are consumed only after failed validation. Exhaustion never creates an out-of-pool point: the current coefficients are returned with `computed_with_warning`, `precisionTargetMet -> False`, and reason `candidate_pool_exhausted`.

Alternatively, `EpSampleAngleRange -> {thetaMin,thetaMax}` selects an open angular interval in radians. Up to three uniformly spaced interior rays are used, while magnitudes remain controlled by the same precision formula and have no user radius cap. Validation points retain their corresponding angles and reduce only the radii. The high-precision real and imaginary parts are rationalized to exact values in $\mathbb{Q}(i)$ before MadStree performs its exact dlog pullback. The

option is mutually exclusive with explicit `EpSamplePoints`; `Automatic` keeps the original positive-real-axis schema. All related options default to `Automatic`, preserving the original default call and adapter JSON field set.

Ordinary coordinate-value rows are saved; a transient waypoint is written as {{`value1,value2,...`},`"tmp"`}. Ordinary saved points in the `MSEvaluatePath` result carry coordinates, the full master-integral vector, status, and user index. Use `MSExportEvaluationData[result, MSOutputDirectory -> outputDirectory, ExportFormats -> {"CSV", "JSON"}]` to export them. The option accepts `Automatic`, either single format, or a list of the two. Status is `written` only when every requested format succeeds. Precision and relative-difference fields that are already strings round-trip through JSON without an additional layer of literal quotes. Transient waypoints are excluded from the ordinary-point export.

FlintNDE classifies the local singularities and runs a capability preflight before starting; the current generic boundary producer of MadStree delivers $\{a, b, C\}$ only when the pulled-back connection is an exact regular-singular one, so the addition of $\{\phi, a, b, C\}$ is a backend path-saving capability rather than a change of the dS boundary source. If the input requires higher-order or irregular singularities beyond the declared capabilities of the backend, the adapter fails closed directly and does not disguise ordinary Taylor transport as a singular solution.

In the original-coordinate formulation, the $e^{-K_v t_v}$ damping of the integrand looks exponentially asymptotic at $K_v \to \infty$; this does not mean that the dlog system of the normalized master integrals necessarily has an essential singularity at the blow-up coordinates $x_j = 0$. After extracting the known physical weights of Eq. (158) and performing the nested blow-up, the currently accepted exact systems must be ordinary matching problems or Fuchsian regular-singular problems. If the actually pulled-back system still has an irreducible pole of order higher than two, needs Stokes/ramification, or is not in the current exact field, the capability preflight returns the actual classification and stops; the package does not automatically interpret it as a solved boundary condition.

The following are already checked end to end: the [2] boundary for a single vertex with three massive propagators, the analytic chamber boundary for the two-vertex pure-massless case, the generic 15-dimensional boundary and transport for the three-vertex massive/massless case, and the five-dimensional G_{++} two-vertex system. For the pure-massless defining integral, both DE residuals of the contact-atom weights and the complete target vector are checked; for the massive five-dimensional system, the connection, the singular pullback, the five leading vectors, the five coefficients and the target vector all agree with an independent paper-based route. Structural checks also cover common-theta simultaneous events, massless/massive coincident quotients, the per-propagator equal-time mean of the boundary coefficients of coincident massive (anti-)time-ordered propagators of Eq. (165), with arbitrarily many of them joining the two vertices, and the generic boundary/residue of the triangle time-only loop graph. The FlintNDE adapter additionally checks ordinary/regular-singular save points and the fail-closed behavior for super-capability poles. Representative run evidence is not a point-by-point certification of all topologies and parameter regions; the generality comes from the same metadata data flow and the joint regression over structurally different topologies, not from an enumeration of dedicated production branches.

13 Public interface reference

The table below lists every public symbol of the current version in the order in which a complete calculation calls them: initialization, sectors and master integrals, iterative reduction, analytic matrices and artifacts, boundary data, numerical transport and the runtime environment. Each entry gives a one-sentence purpose; derivations, gate conditions and return fields are expanded in the indicated sections, and the table and the usage strings of `Kernel/MadStree.wl` share one source. Every entry returns a structured `Failure` carrying a reason for invalid input and never degrades silently.

Table 1: Complete public interface of MadStree, grouped in call order.

symbol	purpose
Initialization and context	
`MSInitTree[spec]`	initializes a tree topology and returns a context; every massless (anti-)time-ordered line uses the shared two-state quotient (Sec. 1)
`MSInitTimeGraph[spec]`	initializes a time-only graph with cycles from the same vertex/propagator fields, without loop momentum or ISP
`MSInitVertexFamily[spec]`	initializes a single-vertex function family without graph topology, from compact `ki/nui` or explicit `hBlocks/exponentialBlocks` input (Sec. 11.4)
`MSContextQ[obj]`	tests whether an object is a valid MadStree context; the shared input gate of the public entries
option `NuConvention`	selects s_ν of Eq. (2) at initialization; every interface reads it from the context afterwards, with no per-call override
Sectors and master representation	
`MSSectors[context]`	the deterministic order of contact-reachable sectors, with sector keys, pinched propagators and master counts
`MSSlotRegistry[context, sector]`	the graph-wide two-state slot registry of a sector, the authority for the `stateBits` order; accepts a sector key or `All`
`MSIntegral[s, n, a]`	the normalized master $J_s(\boldsymbol{n};\boldsymbol{a})$ with the domain of Eqs. (61) and (62)
`MSBareIntegral[s, n, a]`	the inert bare integral $\mathrm{I}_s(\boldsymbol{n};\boldsymbol{a})$ with the same three indices, Eq. (127); display and audit only, never recurrence, DE or transport
`MSIntegralDefinition[integral, context]`	single-object query returning the bare label, the exact $\mathcal{N}_s$ and the inert identity (Sec. 11.3)
`MSMasterIntegrals[context]`	master records in matrix order, each carrying `integral/bareIntegral/normalization/definition`
Iterative reduction and contacts	
`MSRecurrenceStep[integral, component, context]`	one step moving a nonzero shift toward zero, a single application of Eq. (130) or (131); with the component omitted it takes the first nonzero shift, and it refuses steps that cross a divergent layer (Sec. 11.5)
`MSReduce[expr, context, opts]`	the complete iterative reduction: a scalar returns one ordered coefficient vector, lists and nested tables preserve their shape with per-element reports (Sec. 11.4)
option `MasterBasis`	the master ordering of the reduction output; must be a permutation of all context masters without omissions or repeats
`MSContactMaps[context, sector]`	per-event injection matrices by component, target sectors, target shifts and normalization ratios, corresponding to Eqs. (64)–(78)
Analytic matrices and artifacts	
`MSFormulaMatrices[context, sector]`	per-component M_1, M_0 and energy letters, with the sector diagonalizer `U` and `UInverse`
`MSDLogDE[context]`	the ordered masters with the block-triangular dlog connection, letters, residual and certification status; the closure check calls `MSReduce` internally (Sec. 9)
`MSFormulaData[context]`	the aggregate of all-sector masters, per-sector recurrence metadata and the complete dlog DE
`MSWriteFormulaArtifacts[context]`	writes the aggregate as `masters.wl`, `recurrence_metadata.wl`, `dlog_de.wl` and `manifest.wl` below `results/madstree_formula/run-UUID` by default and returns every actual path
options `TimePowerRules` / `MSOutputDirectory`	vertex time powers enter the recurrence metadata only, never the analytic dlog DE / the artifact output directory
Boundary and blow-up certificates	
`MSBoundaryData[context, targetRules]`	a finite starting vector in the dlog master order from the $k_0 \to \infty$ Frobenius formula; `WorkingPrecision` defaults to 200 digits and unsupported cases fail closed

Table 1 (continued)

symbol	purpose
`MSBoundaryChartCertificate[context, targetRules]`	the machine certificate of nested blow-up, per-sector theta fixing and normal crossing; `RankOrder -> All` checks all strict charts
`MSBlowupCoordinate[i]`, `MSDampingEnergy[v]`	the local coordinate x_i and damping energy K_v symbols appearing in certificates and boundary records (Sec. 12)
options `RankOrder` / `BoundaryScale` / `BoundarySeriesOrder`	energy rank order / finite anchor scale / boundary series truncation (Sec. 12.6)
Numerical transport, regulator and export	
`MSNumericalSystem[de, spec]`	validates the numerical substitutions and boundary vector and constructs the numerical DE data
`MSEvaluatePath[context, pointSequence, opts]`	single-stage multipoint evaluation: boundary initialization and full-sequence transport in one backend process (Sec. 12.6)
`MSReconstructEpSeries[context, ep, pointSequence, opts]`	Laurent reconstruction in the common regulator: certified lowest power, automatic sampling and incremental fitting (Sec. 12.7)
`MSExportEvaluationData[evaluation, opts]`	exports the saved ordinary-point records of one evaluation to CSV/JSON; transient waypoints are excluded
numerical and regulator options	`WorkingPrecision`, `TransportOrder`, `ReferenceTransportOrder`, `TargetRelativeError`, `FlintNDEPathPlanning`, `SingularityMode`, `ParameterRules`, `ParallelTaskCount`, `MessageLanguage`, `MaximumEpPower`, `EpGoalDigits`, `EpFitExtraOrder`, `EpFitOrderIncrement`, `EpFitMaximumRounds`, `EpSamplePoints`, `EpValidationPoints`, `EpInitialInternalMaximumPower`, `EpSampleAngleRange`; defaults and gates in Secs. 12.6 and 12.7
options `ExportFormats` / `SignificantDigits`	export format selection and exported significant digits
Runtime environment	
`MSFlintNDEConfiguration[]`	returns the current version directory, the built-in FlintNDE relative path and availability
`MSSetFlintNDERelativePath[path]`	changes the single version-directory-relative path to the FlintNDE package
option `MSRuntimeDirectory`	runtime root; `Automatic` writes `results_temp` beside the calling script, an explicit path is used directly
option `PythonExecutable`	the backend Python command; the default reads the `MADSTREE_PYTHON` environment variable and otherwise `python` from PATH

Initialization itself writes no files. The four analytic artifact files above are written and reused only when `MSWriteFormulaArtifacts` is called explicitly or before a numerical entry starts the backend, with the actual paths returned under `formulaArtifacts`; the numerical phase constructs a local parameterized DE inside the function and never overwrites the formal analytic artifacts. Runtime temporaries (requests and logs under `nde/`, successful caches under `cache/`) live under the root set by `MSRuntimeDirectory` and never enter the package source directories. The seven distributed `Examples` scripts cover the main workflow (Sec. 14).

14 Usage examples

14.1 Single-vertex family: definition, reduction, and unified point input

Use `MSInitVertexFamily` for a function product at one vertex instead of constructing a fake one-vertex topology. Package example `Examples/02_single_vertex_family.wl` gives both the compact and equivalent explicit models, reads the masters and dlog DE, and reduces one shifted integral. Its numerical section uses only one entry:

```
parameterRules = {q->1,nu->1/2,a->2};
singlePointSequence = {{k0},{3 I}};
```

```
multipointSequence = {{k0},{3 I},{7 I/2},{4 I}};
single = MSEvaluatePath[ctx,singlePointSequence,
  ParameterRules->parameterRules];
multi  = MSEvaluatePath[ctx,multipointSequence,
  ParameterRules->parameterRules];
```

The example compares the common first point component by component, demonstrating that a single point is merely a length-one `pointSequence`.

14.2 Two-vertex massless (anti-)time-ordered propagator: multiple sectors and points

`Examples/01_massless_time_ordered_edge.wl` starts from two `+` vertices and one public `"massless"` propagator. MadStree derives the T/quotient classification, the top sector and its subsectors, masters, contact map, and dlog DE. The example uses {`k1,k2`} as the header, puts only coordinate values in later rows, and supplies common `q/a1/a2` values once through `ParameterRules`. It then enables FlintNDE planning and fast multipoint evaluation and exports saved points with `MSExportEvaluationData`. A transient waypoint is {{`value1,value2`},`"tmp"`}; there is no separate path object.

14.3 Massive three-vertex tree: zero middle external-leg energy is not automatically singular

`Examples/05_massive_three_vertex_tree.wl` initializes one context for a three-vertex tree with two massive propagators. It fixes $k_3 = 5\mathrm{i}$, $q_{12} = 1$, $q_{23} = 2$, $\nu_{12} = 3/4$, $\nu_{23} = 1/3$, and $a_1 = a_2 = a_3 = 1$. The original sequence

```
{{k1,k2},{8 I,2 I},{7 I,4 I},{6 I,3 I},{9 I,5 I}}
```

is retained. A second independent `MSEvaluatePath` call plans, evaluates, and exports

```
{{k1,k2},{8 I,0},{7 I,0},{6 I,0},{9 I,0}},
```

so the two runs share only the context.

The singularity test substitutes each point into the complete top/contact-sector dlog letters; it does not test whether an individual coordinate is zero. No letter vanishes at these four points, so all are ordinary: all 25 master-integral components are saved, the `singularityClassifications` table is empty, and the maximum pointwise relative difference between the lower-order primary chain and the higher-order reference chain is about 2.31×10^{-31}. If another family really meets an exact letter at zero or at any nonzero complex coordinate, the same FlintNDE local power–log entry classifies it as removable, pole, or log divergent without dispatching on a variable name or graph type.

14.4 Three-vertex regulator: automatic Laurent support and incremental fitting

`Examples/06_massless_three_vertex_ep_regularization.wl` sets $a_1 = -1 + \epsilon$ and $a_2 = a_3 = 1 + \epsilon$, while the user requests only `MaximumEpPower->0`. Before the numerical stage the example runs two symbolic self-checks: `MSReduce` acts once on the sum of two integrals whose powers lie one unit below the master powers (with status/residual/remaining/singularLayers verified); the same reduction is repeated on a context re-initialized with the regulated powers as `timePower`, so the coefficients carry an explicit ϵ. The package then certifies the lowest power from the symbolic boundary and DE and then selects production and independent validation points. If validation misses the target, only the internal fit order and new points are added, while previous values are reused. Its coordinate table contains only ϵ-independent k_i values, while $a_i \to c_i + \epsilon$ occurs only in `ParameterRules`. This example certifies lowest power -1 and therefore returns the ϵ^{-1} pole coefficient together with the finite part.

14.5 No physical external leg: private auxiliary energy to zero

`Examples/07_zero_external_leg_energy.wl` shows the two equivalent inputs for a vertex without a physical external leg: omit `externalLegEnergy` or set it explicitly to zero. The example puts only physical coordinates in `pointSequence`, verifies that the private auxiliary coordinate remains absent from public input, and transports it from the damping boundary to zero while preserving the same analytic dlog system.

15 Minimal algorithm for direct formula generation and its acceptance tests

Without running a sampled IBP or reduction, direct formula generation proceeds as follows:

1. Build the full-graph building-block registry from the topology: a two-dimensional h slot for each massive propagator attached to a vertex and one shared two-dimensional slot for each unpinched massless (anti-)time-ordered propagator; also store the slot incidence list of each vertex. Theta-free exponentials do not enter the registry and only give the scalar p_0 of each component through Eq. (58).

2. Fix S_e, P_e of Eq. (30) and the incidence of the two vertices for each massless (anti-)time-ordered propagator; Wightman/external exponents do not create any slot.

3. Assemble the Kronecker sums vertex by vertex according to Eqs. (45)–(46); the one-dimensional exponential terms and all two-dimensional terms use the same embedding API, and ς_v, σ_e and ϵ_{ve} are kept strictly separate.

4. First verify Eq. (49), then enumerate the diagonal elements of Eq. (52) with one global U_s; invert only the scalar diagonal elements of M_1 and $\widetilde{\mathrm{M}}_0$.

5. Run an event BFS over the current-component bundles according to Eq. (82); construct the S_s, P_s quotient for coincident spectators, and fix the subsector powers with Eq. (83).

6. Output the general-index one-step recurrences with Eqs. (68)–(71); append the lower-sector sources with the event-level contact selector, and return the $\mathrm{M}_1/\mathrm{M}_0$ singular layers by direction.

7. Output the block-triangular dlog candidate directly with Eqs. (87) and (90); nonzero subsector shifts are reduced back to the global masters automatically by Eq. (99), and then Eq. (132) is applied.

8. Call `MSBoundaryChartCertificate` for the chosen or all root ranks; only when the complete connection, normalization, Jacobian, theta fixing and quotient all pass is the automatic boundary entered.

The basic acceptance tests are:

- the $4 \to 2$ rank of a single massless (anti-)time-ordered propagator, $P_e S_e = \mathbb{I}_2$, Eqs. (32)–(33), and the contact vector;
- in a two-vertex graph with one propagator, the same shared bit has opposite signs in the two κ_v;
- the four common eigenstates of the two shared slots in a three-vertex chain, rather than an independent enumeration per vertex;
- the global $U_s \mathrm{M}_{0;v,s} U_s^{-1}$ is fully diagonal for a mixed massive/massless tree;

- the formula one-step relations equal the naive time-IBP term by term in the exactly same ordered normalized basis;
- the formula DE equals the naive DE matrix element by matrix element, all lower-sector residuals are empty, and exact flatness is passed;
- in the mixed three-vertex case, the five 15×15 matrices obtained from the public `ds` of dSIBP through the slot-by-slot adapter and `MSReduce` are strictly equal to the direct formula DE;
- the physical output basis additionally contains Eq. (109), without missing k, k^2 or the normalization derivative.

16 Conclusions

The purely massive formulas are Kronecker sums on the building-block registry: the theta-free exponential of each vertex gives $\Delta\mathrm{M}_1 = 0$ and $\Delta\mathrm{M}_0 = \mathrm{i}p_{0,v}\mathbb{I}$; each two-dimensional h block of a massive propagator gives $\Delta\mathrm{M}_1 = -(1-2\nu_i)P_1$ and $\Delta\mathrm{M}_0 = -\mathrm{i}k_i\sigma_2$; all terms carry the identity matrices of the other slots. Here ν_i is the parameter after Eq. (4), and the default positive-prefactor convention of MadStree requires no further replacement. Positive and negative vertices only change the sign ς_v of the vertex exponential; along $k_{0,v} = -\mathrm{i}\varsigma_v K_v$ with $K_v > 0$, both become $e^{-K_v t_v}$ and the two-dimensional h atoms are unchanged.

A massless (anti-)time-ordered propagator with theta first retains the same two-state factors at its endpoints as in the massive case, and S_e, P_e then reduce the four states to one shared two-state. The two vertices no longer carry independent factors of that propagator; they act on the same slot with opposite signs. The vertex-by-vertex tensor factorization therefore fails, while the full-graph Kronecker sum, the common diagonalization, the iteration matrices and the dlog DE remain directly available. The theta derivatives only add the lower-sector block generated by $|1\rangle_e$. In the reference h-state $\mathrm{I}_{11} = +\mathrm{I}_{00}$, whereas in the package quotient $\mathrm{I}^J_{11} = -\mathrm{I}^J_{00}$; the two are related exactly by the constant D_{σ_e} and are not interchangeable.

The single-vertex recurrences are not obstructed by the shared slot: summing the blocks of the vertex's incidence list gives $\mathrm{M}_{1;v,s}$ and $\mathrm{M}_{0;v,s}$. Different vertices share the state bit on the same massless slot and carry opposite vertex signs. Each two-dimensional slot receives a single fixed Pauli direction, so operators on the same slot commute pairwise; the Kronecker product of the local 2×2 diagonalizers reduces the large-matrix inversion to a letter-by-letter reciprocal. If non-collinear Pauli directions ever occur on one slot, this fast path no longer applies. Lowering fails only when an M_1 eigenvalue vanishes, and raising only when an energy letter of M_0 vanishes; a matrix that is not inverted in a given direction produces no pole in that direction even if it vanishes.

The dlog DE follows directly from the same registry: the sector diagonal blocks use the commonly diagonalized $\log\kappa$ with $\mathrm{M}_1(A+1)$, the top-to-sub blocks use the same $\log\kappa$ times the $R^{(1)}$ contact map, and the sector normalization is applied at the end. A large IBP system is therefore not needed as an intermediate step. The output has dlog form only after the normalization ratio, constant residue, block closure and flatness all pass.

The boundary likewise uses the initialized $K_v = \mathrm{i}\varsigma_v k_{0,v}$ and does not permanently rename the user variables to `pik0`/`mik0`. The preferred ordinary point selects the strict chart by descending $\mathrm{Re}\,K_v$, equivalently by ascending $|\tau_v|$, since a large damping energy supports only a smaller typical $|\tau_v|$. This choice shortens the first continuation segment and reduces the letters that may need to be bypassed; it does not replace the ordinary-point and normal-crossing certificates.

The dimensionless form of a massless building block is $h_{zz} + h = 0$. The physical form $\partial_\tau^2 h = -k^2 h$ is equivalent, differing only by the two chain-rule steps of $\partial_\tau = -k\partial_z$, and requires no separate local EOM. This preserves the 2×2 structure of the reference formulas and recovers k^2 and the contact correctly in the final physical basis.

References

[1] Jiaqi Chen and Bo Feng. Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations. *JHEP*, 06:199, 2024. doi: 10.1007/JHEP06(2024)199.

[2] Jiaqi Chen, Bo Feng, and Yi-Xiao Tao. Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations. *JHEP*, 03:075, 2025. doi: 10.1007/JHEP03(2025)075.